PHY3009
NUMERICAL INVESTIGATION OF DIFFERENTIAL EQUATIONS AND STOCHASTIC SYSTEMS

TOM FIELD

ABSTRACT. This synopsis covers differential equation as part of PHY3009. The aim of this course is to gain the programming and analytical skills required to solve differential equations relevant to physical systems. There is also the option to work on cellular automata.

Note that module marks are based 100% on continuously assessed assignments. Therefore, it is critical that you hand in all assignments promptly. You are strongly urged to participate in all sessions.

© 2026 Tom Field - All rights reserved

CONTENTS

1. Introduction 4
2. Web Resources for Programming 5
3. Python tutorial - Information 6
3.1. Information 6
3.2. Exercises 6
4. Python tutorial - number crunching
Maths, Decisions, Loops and Functions 14
4.1. Basic mathematical functions 14
4.2. exercises 14
4.3. Scientific mathematical functions 15
4.4. exercises 16
4.5. Decisions; if, elif, else 16
4.6. exercises 18
4.7. Loops: while and for 20
4.8. exercises 21
4.9. Functions 23
4.10. Exercises 24
5. Python tutorial
Lists, Tuples, Arrays and Data Handling 25
5.1. Lists, tuples and arrays in Python 26
5.2. Exercises 26
6. Python tutorial - libraries 31
6.1. Types of library 32
6.2. exercises 32

Date: September 18, 2026.

7. Introduction to cellular automata and Numpy arrays 35
7.1. Arrays using the Numpy library 35
7.2. A one dimensional cellular automaton 37
7.3. The game of life 38
8. Pandas data library 39
9. Numerical solution of ordinary differential equations Part I 40
9.1. introduction 40
9.2. Radioactive decay. 40
10. Introduction to Ordinary Differential Equations 43
11. Numerical solution of ordinary differential equations part II 46
11.1. Euler Method 46
11.2. Euler-Cromer Method 46
11.3. Runge-Kutta Method 47
11.4. Convergence 47
11.5. Symplectic integrators 48
11.6. Adaptive timestep integrators 48
12. Ordinary Differential Equation - Practical Exercise 49
13. Class Assignment 1 - Ordinary Differential Equations 50
13.1. Set up 50
13.2. Tasks 50
13.3. Write up 50
14. Class Assignment 2 - Partial Differential Equations 51
14.1. Set up 51
14.2. tasks 51
14.3. Write up 52
15. Projects 53
15.1. Notes for all projects 53
15.2. Project guidelines 53
15.3. Word limit 54
15.4. Project mark scheme 54
16. Project Details 56
16.1. Investigation of damped/driven oscillation including chaotic motion of a damped/driven pendulum. 56
16.2. Motion of astronomical bodies in the system including two-body, three-body and n-body problems, and Kirkwood gaps. 57
16.3. Shooting methods for solving the Schrödinger Equation in one dimension. 60
16.4. Electric potential calculations including determination of the potential around a dipole. 63
16.5. Waves on a string including simulation of the propagation of a wave along a string and standing waves. Could include the use of an FFT - Fast Fourier Transform - to

analyse the frequency components of the wave on the string. 64
16.6. Percolation; investigation of open lattices where some sites are filled and some empty 66
16.7. Percolation - immunization and disease spread 67
16.8. Protein folding including simple two dimensional simulations; could include three dimensional folding. 68
16.9. Your own choice! This must be discussed prior to approval. 70
17. Reference tables 72

1. INTRODUCTION

(return to contents)

The aim of this course is to learn how to simulate dynamical systems by solving ordinary differential equations numerically. There is also the option to study systems which can be modelled stochastically, *viz.* with a model that relies on random number generation.

The course consists of:

- Introductory exercises to practice programming.
- Assignment 1 - a formative assessment to introduce the use of numerical methods to solve ordinary differential equations.
- Assignment 2 - a formative assessment to introduce the use of numerical methods to solve partial differential equations.
- Assignment 3 - a project. You can choose from a number of different systems to investigate.

The work divides into two parts. The first part is an introduction, which includes the first two short assignments. In the second part of the work everyone works on a project, which is the major assignment for this part of PHY3009. It is worth beginning to think about which project you would like to work on during the first part of the course.

Note that this guide is written with hints for the computational challenges in the Python programming language, which will be used in the first part of the course. In the second part of the work when working on the project there may be some freedom to work in a different programming language, but discussion is required to obtain approval to work in a different language.

2. Web Resources for Programming

(return to contents)

There are many online tutorials and guides.

For the actual programming the online IDE (interactive development environment) that we will be using is at

- `https://repl.it`

The advantage of this website is that it is possible to share a link to you programs with other people so that two people can work on it at the same time. This will be helpful for demonstrators to be able to review code online.

Note please take regular copies of your code from `repl.it` to keep backupss as occasionally `repl.it` has crashed and people have lost code, but overall it has been stable.

Everything in this course should be possible on `repl.it`. Note, though, that another way of working is to program on your local computer with, for example, Spyder or Jupyter Notebooks, which can be downloaded as part of the Anaconda package. There are two important points to consider if you program on your local machine. First, it is still important to keep backups on, for example, Onedrive or another cloud location. Secondly, it is important to be able to use `repl.it` so that you can copy and paste your code there when you want to discuss your code with a demonstrator.

3. PYTHON TUTORIAL - INFORMATION

(return to contents)

The following notes are a tutorial introduction to Python for this course. The notes are not as extensive as a text book, but there are lots of practical exercises and challenges for you to work through.

3.1. **Information.**

The I of IT is information, and the first part of this course is to learn to store information in computer memory and to retrieve it.

In programs, **information can be stored in variables**. To be more accurate, variables are labels to help with storing and retrieving information from memory. There are different types of variables, which include

- Integers; `0, 1, 2,... -1, -2,...` Positive and negative integers up to a high limit.
- Floating point numbers, such as `1.234567e89`, which means 1.234567$\times 10^{89}$. For example, `3.1415927e00` is 3.1415927$\times 10^{0}$. Now, $10^0 = 1$, so the number is 3.1415927. Note that it is not necessary to inlcude the `e` if it not required; 3.1415927 can also be represented by `3.1415927`. Other examples; `3.243e03` $= 3243$; `6.392e-4` = 0.0006392 = 6.392×10^{-4}; `2.65e06` = 2650000.
- Complex numbers, such as `2+3j`.
- Boolean; boolean variables have a value that is either `True` or `False`. Note that the first letters of `True` and `False` must be capitals.
- A 'string' of characters such as `"this is an example of a string"`. Note that when strings are written in a program they are enclosed by double quote; `"`. In fact, single quotes, ', can also be used. Finally, if you use triple quotes, `"""` or `'''` at each end the string can extend over multiple liness. Whenever `"` is used in this document ' can also be used.
- A 'file object'. A variable can give a program access to a file for reading in data from a file, or saving data to a file.
- Many other types including; list, tuple, dictionary and set.

Note that in many programming languages it is necessary to declare variables and indicate what type they are before they are used, but this declaration is not required in Python. In Python, variables are 'declared' by starting to use them.

We will be using keyboard entry and output to the screen to pass information to the computer and to get information back from it. We will also be loading in information from files and saving information to files.

3.2. **Exercises. In programming it is best to learn by doing**, so there are programs to write in the following exercises.

It is recommended that you save your programs at the end of each exercise and start each exercise with a new program so that you can review your programs later. One way of saving you programs is to copy them all into a document, like an email, a Word document or a Google

document or a text file. Wherever you keep them then do back up the file so that you have a copy if the original file gets lost. Sometimes copying and pasting files from an email/Word document/Google document/text file may lead to issues with characters changing or issues with indents; indents are described below. So you may want to test what works best for you.

One reason to save your programs is that online engines like repl.it can crash and this can lead to code loss.

Note that you can copy bits of code from the electronic copy of this document and paste them into your programs. Mostly (?always) the code should just work, but there is a risk that sometimes odd characters may not copy correctly and may need to be corrected. When the text is in `this type writer font` then copy and paste should work ok. There can be an issue with *indentation* so it is important to check you code has the correct indentation when you copy and paste it.

It is most important that you work at your own pace and understand what you are doing. Ask for help when you need it.

AIMS - Information

The aims for this section are;

- to become familiar with different types of variable,
- to print information to the screen,
- to read in information from the keyboard,
- to write information into files and
- to read information from files.

Note that everyone will be working at different speeds depending on how much prior experience they have and other factors. Please don't compare your own progress to the progress of others. The aim here is for everyone to learn at their own pace so that everyone can understand what they are doing as they work through the course.

(1) ☐ *Create a Python program*

You can copy the line below from the electronic version of this document and paste it to make a vey simple python program that simply prints 'Hello World!' on the screen.

```
print("Hello World!")
```

---

(2) ☐ Change the program from Exercise 1 to print a different message onto the screen, such as 'Hello world again!'.

---

(3) ☐ Write a program which assigns two integers values to variables `n1` and `n2` and then prints them to the screen. The following pieces of code may be useful

```
n1=5
n2=42

print (n1, n2)
```

---

(4) ☐ Change your program from exercise 3 so that it prints to the screen

```
Please may I have 5 hot dogs and 42 burgers
```

with all the text on one line and where the numbers come from the variables `n1` and `n2`.

Note that you can print several things together separated by commas with the `print` statement. For example,

```
n1=5
n2=42
print("n1 is equal to", n1,"and n2 is equal to", n2);
```

will print on the screen

```
n1 is equal to 5 and n2 is equal to 42
```

---

(5) ☐ Change your program from exercise 4 so that it prints to the screen

```
Please may I have
5 hot dogs and
42 burgers
```

where again the numbers come from the variables `n1` and `n2`.

Note that there are (at least) two ways to do this exercise. You could use three different print statements. You could also use one print statement, but include a special control sequence, `\n`, which 'prints a newline'. See if you can get the output required using both of these methods. Note that the output does not need to match the output above exactly; the important point is to get things written on three lines rather than a single line.

---

(6) ☐ Modify your program from Exercise 5 so that it prints

```
Please may I have
19 hot dogs and
15 burgers
```

by changing the values of `n1` and `n2`.

---

(7) ☐ Modify your program from Exercise 6 so that you can enter from the keyboard the number of hot dogs and burgers that you want.

In Python a useful function for keyboard entry is `input()` where a message can be put in the brackets. Do for example if you want your program to ask the user for their age you can use the following statement;

```
age = input("What is your age in years?")
```

---

(8) ☐ Write a program which calculates the average of four numbers `d1`, `d2`, `d3`, `d4`. Assign values and use mathematical functions to calculate the mean average, `average`. The program should assign values to `d1` `d2`, `d3` and `d4`, with `d1=3.7` and similar statements. Calculate the average and assign this value to variable `average`. Finally print out the values of `d1`, `d2`, `d3`, `d4` and `average`.

The following line of code can be used to find the average;

```
average=(d1+d2+d3+d4)/4
```

Note that in Python we can add, subtract, multiply and divide in equations with the symbols `+ - * /`. Furthermore, note that brackets, `( )`, can be used to ensure that the result of calculations inside the brackets are calculated with the highest priority as, for example, in the equation above for `average`.

---

(9) ☐ Modify the program in exercise 8 above to read the values of `d1`, `d2`, `d3` and `d4` from the keyboard using `input()`. For example, to read a value into `d1` use:

```
d1=float(input("first number?"))
```

Note that we need to use `float( )` to convert the 'raw input' from the keyboard into a floating point number. The 'raw input' from the keyboard is a string.

---

(10) ☐ Write a program which accepts two integer numbers from the keyboard using `input()`, and then prints out their difference, sum and product. To convert the raw keyboard input into an integer you will need to use `int( )` thus;

```
n1=int(input("first integer?"))
```

Note that you may find the function `abs()` useful here. `abs()` returns the 'magnitude' of a number; so for example `abs(-4)` will give the output of 4. Similarly, `abs(4)` also gives the output of 4.

---

(11) ☐ Write a program where the user enters a message from the keyboard and the computer then prints it on the screen. So far using the `input()` function we have converted the text input into numbers; integers and floats. Now we will leave the 'text' entered as a 'string' that is to say a series of characters.

```
string=input("Enter some text!")
```

---

(12) ☐ Now write a program where three strings are entered, added together and then printed on the screen. Note that in Python to add strings together you can do something like

```
added_string=string1 + string2 + string3
```

---

(13) ☐ Now modify your program from exercise 12 so that 'newlines' are included in the combined string between each of the three strings entered from the keyboard. To do this you will need a 'special' character combination that gives a newline; `\n`. Note that `\` is a 'backslash' and not the same character as `/`, a forward slash, which is used for division in equations. Remember you can add strings with `+` so you will need to add newline characters with `+"\n"+` when you are combining the three strings from the keyboard.

So to summarise; in this exercise three strings should be read in from the keyboard. They should then be combined with newline characters to form a single combined string. When this combined string is printed by the computer it should give the three strings entered from the keyboard on three separate lines.

---

(14) ☐ In this exercise the aim is to modify one or more of your existing programs without changing the way they work. The first modification is to see how to put more than one command on one line, the second is to add comments to your program.

- You can have more than one command on a single line by using a semicolon as, for example, below when setting up the values of several variables;

  ```
  d1=1; d2=3; d3=3.7; d4=3.2
  average=(d1+d2+d3+d4)/4
  ```

  try putting some commands together on the same line with one of the programs that you wrote in an exercise above and check that it still works after you have done it.

  It may be helpful sometimes to put commands together on a single line in this way, but try to lay out your code so that it can be easily understood.

- Add comments to your program. In Python anything on a line after a # (hash) character is ignored. **It is good practice to put comments into your program** to remind yourself and to explain to other people what different parts of your program do. For example,

```
# set up four numbers to find the average of
d1=1; d2=3; d3=3.7; d4=3.2
# find the average
average=(d1+d2+d3+d4)/4
```

---

(15) ☐ Now modify your program from exercise 13 so that the three strings entered from the keyboard are saved in a file. The strings can be entered in the same way as before, but saving to a file is new.

To save to a file it is necessary to open the file, write to the file, and close the file; for example,

```
file_out = open("text_output.txt","w")
          # the "w" means write the file
          #    ("r" means read the file)
file_out.write(string1 + "\n")
file_out.close()
```

The program above opens a file named `text_output.txt` and writes a single string to it, `string1`. Note that in the `write` command a newline character, `\n` is added to `string1`.

- Using the program above as a template enter three strings from the keyboard and write them to a file.
- Find the file and open it - are the contents of the file what you expected?
- See what happens if you miss out the newline character and write to the file with `file_out.write(string1)` or similar commands and see how this changes the file that is written.

---

(16) ☐ In exercise 15 the example code has `file_out` as a variable name to reference the file being written. Other names can be chosen instead. For example, the Python code below

```
fout = open("text.txt","w")
fout.write("message")
fout.close()
```

uses the variable `fout` to create a text file called `text.txt`, write `message` in it, and then close the file.

Change the variable name for the file in your code from exercise 15 and rerun the program to check it still works. Remember you need to change the name eveywhere it is used; for opening the file, writing to the file and closing the file.

(17) ☐ Modify your program from exercise 10 so that it prints out the two numbers, their difference, their sum and their product to a file `data_output.txt` as well as printing to the screen.

Note the write command, `write( )`, must have a string inside the brackets so it is necessary to convert the numbers to strings using `str( )`. The newline character should be added again. So to write a number, `n1`, to a file we can use

```
file_out.write(str(n1)+"\n")
```

Make sure that you can find the file you wrote and open it to check that it is what you expect.

Note it is a good idea to save the file with a `.txt` extension. A good filename would be `data.txt`.

---

(18) ☐ The aim of this exercise is to write a program to read in the data from the file saved in exercise 17 and print the numbers on the screen.

Note this time you need to open the same file, but for reading instead of writing, and use `file_in.readline()` to read in the data.

```
file_in=open("data.txt","r")
n1 = int(file_in.readline())
```

Note that the `readline` command reads in a line of the file. Now, if you used newline characters as suggested in exercise 17 then there should be one number on each line and the `\readline` should read in a single number.

Also note that we need to convert characters to an integer in the same way that we had to when entering integers from the keyboard with `int( )`.

---

(19) ☐ Modify your code from Exercise 9 so that the average is printed to a file. Check that you can find the file, and then write a program to read in the average value from the file you saved.

---

(20) ☐ Now write a program to read in a 'string' from the keyboard. In this program the computer should ask you to enter your first name. After it has read in your name then it should print out a message such as 'Hello Tom!'.

---

(21) ☐ Now modify your code from Exercise 20 so that your name is saved in a file. Then write a program to read your name in from the file.

---

(22) ☐ In this exercise you should enter a string for the filename of a file and then save something to a file with the name your entered from the keyboard.

Note that if your string is called `file_name` then you can open a file to write with

```
file_out= open(file_name,"w")
```

Furthermore you can also read in a file with a filename you type into the keyboard using

```
file_in=open(file_name,"r")
```

---

(23) ☐ You should be aware that there is another way of opening files in Python using the command `with`. For example;

```
with open("text2.txt","w") as fout:
  fout.write("message")
```

Some would say that this method is better because it is not necessary to write a separate command to close the file after it is written. Instead what happens is the file remains open for all of the command that are **indented** after the `with` statement.

Indentation is very important in Python programs as we will see later.

Optionally you may want to rewrite one of your programs to save a file using the `with` command.

In fact, of course, there are other ways of opening files using functions in libraries such as the `numpy` library.

---

4. PYTHON TUTORIAL - NUMBER CRUNCHING MATHS, DECISIONS, LOOPS AND FUNCTIONS

(return to contents)

One of the main strengths of computers is the ability to make many calculations in a short period of time, which can be referred to colloquially as 'number crunching'. High computation power is very useful, but must be handled carefully because computers are very 'stupid' because they will do exactly what you ask them. It is important to write programs carefully so you 'explain' to the computer *exactly* what you want calculated. To work on this we are going to be thinking about mathematical functions and program flow control. There are two main aspects of flow control; decisions and loops.

This section also introduces functions.

AIMS - Number Crunching

The aim of this section is to become familiar with

- Mathematical operations
- Decisions in computer flow control with statements like `if`
- Loops to repeat sets of instructions with statements like `for`
- Functions

4.1. **Basic mathematical functions.** the following operations are always available; the final values of c calculated in the table below are for initial values of `a` = 5, `b` = 3, `c` = 13

Operation	Example	Description	final value of c
+	c=a+b	c takes value of a+b	8
+=	c+=b	c takes value of c+b	16
-	c=a-b	c takes value of a-b	2
-=	c-=b	c takes value of c-b	10
*	c=a*b	c takes value of a $\times$ b	15
*=	c*=b	c takes value of c $\times$ b	39
/	c=b/a	c takes value of b/a	0.6
/=	c/=a	c takes value of c/a	2.6
**	c=a**b	c takes value of a^b	125
**=	b**=a	b takes value of b^a	243
//	c=a//b	c takes value of *floor division* of a by b	1
//=	c//=b	c takes value of *floor division* of c by b	4
%	c=a%b	c takes value of a *modulo* b	2
%=	c%=b	c takes value of c *modulo* b	1

4.2. **exercises.**

(24) ☐ In this exercise the aim is to work with 'floor division' and 'modulo' operations.

Write a program to read in two numbers from the keyboard and then perform 'integer division'. If the two number entered are 13 and 3 the output should be

```
13 divided by 3 is 4 remainder 1
```

Note that here floor division of 13 by 3 gives 4 and the remainder of 1 can be obtained with the modulo function. Test this with other numbers and check that your program behaves as it should. For example, if the two number entered are 5 and 3 the output should be

```
5 divided by 3 is 1 remainder 2
```

The floor division function `//` and modulo function `%` are both really designed to operate on integers, but see what happens if you enter decimals.

---

(25) ☐ Modify your program from exercise 24 so that, instead of integer division, the power function a^b is evaluated, *viz.* `a` to the power of `b`, where a and b are the numbers that you enter from the keyboard. .

4.3. **Scientific mathematical functions.** Many mathematical functions are available in Python. They are part of a maths *library* of functions. The maths library needs to be added to the code. At the begining of the program we need

```
import math
```

We will think more about libraries below, but note the syntax where each function starts with `math.`, *e.g.* `math.sin(x)` returns $\sin(x)$. Note that trigonometric functions work in radians. The functions `math.radians(x)` and `math.degrees(x)` convert x from degrees to radians and from radians to degrees respectively.

Function	Description	Function	Description
math.sqrt(x)	$\sqrt{x}$	math.pow(a,b)	a^b
math.sin(x)	$\sin(x)$	math.asin(x)	$\sin^{-1}(x)$
math.cos(x)	$\cos(x)$	math.acos(x)	$\cos^{-1}(x)$
math.tan(x)	$\tan(x)$	math.atan(x)	$\tan^{-1}(x)$
math.degrees(x)	$180x/\pi$	math.radians (x)	$\pi x/180$
math.pi	π (=3.141...)	math.e	e (=2.718...)
math.exp(x)	e^x	math.expm1(x)	$\mathrm{e}^x - 1$
math.log(x)	$\ln x = \log_{\mathrm{e}}$ x	math.log10(x)	$\log_{10} x$
math.log(a,b)	$\log_{\mathrm{b}} a$	math.log2(x)	$\log_2 x$

Note that there are other packages which can be 'imported' to programs to provide mathematical functions such as `numpy`.

4.4. **exercises.**

(26) ☐ Write a program that takes an angle in degrees from the keyboard, converts it into radians and then prints out its sine, cosine and tangent values. The output should be something like

```
sin(30)= 0.49999999999999994
cos(30)= 0.8660254037844387
tan(30)= 0.5773502691896257
```

Remember to import the maths library with

```
import math
```

---

(27) ☐ extend your program in exercise 26 so that it also prints ln, $\log_{10}$, and $\log_2$ values of the angle.

4.5. **Decisions; if, elif, else.** It is critical in all programming languages to be able to make decisions and act on them. In Python this is done with `if`, `elif` and `else`. For example,

```
if a < b :
  print("a is less than b")
elif a > 10*b :
  print("a is larger than 10b")
else :
  print ("a is between b and 10b")
#check indentation if you copy and paste this code
```

Here if a$<$b then the first message is printed and the `elif` and `if` commands are ignored. If the first logical state is false then the computer moves to the next command which is `elif`, which means 'else if'. If the logical expression for the `elif` command is true then the second message is printed and the `else` command is ignored. Finally if both the `if` and `elif` logical tests were both false then the computer moves to the `else` statement. The `else` means that the commands after the else will always be followed if the previous logical tests are all false - and so the third message will be printed if the previous two messages are not printed.

Note that

- There can be just a single `if` statement without `elif` and `else` statements.
- There can be any number of `elif` statements after an `if` including zero.
- There can only be one `else` statement
- The full colon, `:`, at the end of the `if`, `elif`, and `else` commands must be present
- There can be several instructions to perform after the `if`, `elif`, and `else` commands. They must all be indented.

- If for some reason you do not want to have a command after an the `if`, `elif`, and `else` command then you can use the `pass` command, indented. If there is no command indented then there will be an error.

Some more examples;

```
if a < b :
  print("a is less than b")
  print("so we will make a larger than b")
  a=abs(a)+b

print("this is always printed because these ")
print("two prints statements are not *indented*! ")

# end of if without elif or else
```

Note the importance of indentation in the example above. The indented code after the `if` only runs if the condition is true whereas the code below which is not indented always runs because the `if` condition 'runs out' at the end of the indentation.

The code below has an example of using `pass`. Note also there is no `elif` in the example below.

```
if  a > 10*b :
  pass  #pass means do nothing
else :
  print ("a is less than or equal to 10b")
#check indentation if you copy and paste this code
```

If there are only single operations then `if` statements can be written on a single line. For example;

```
if i<7 : i+=10
```

An essential part of `if` commands is the logical test. It was noted above that the boolean type of variable can have two values; `True` and `False`. We can actually see how logical statements work by entering the following commands;

```
print (4<3)
print (7>1)
```

which should give the output

```
False
True
```

because 4 is not less than 3, but 7 is greater than 1. The computer decides what action in `if` statements on the basis of whether the logical expression is `True` or `False`.

The following operations can be put in logic statements;

Operation	Description	Operation	Description
<	less than	<=	less than or equal to
>	greater than	>=	greater than or equal to
==	equal to	!=	not equal to
and	logical and	or	logical or
not	logical not	(...)	establish priority

Note that logical expressions can be joined by `and` and `or`. If `not` is put in front of a logical expression then it changes `True` to `False` and `False` to `True`. Finally, brackets can be used to establish hierachy.

4.6. **exercises.**

(28) ☐ Bracket position change is important for the following logical expressions;

```
(4>3 or not 12>=3) and 4==3
4>3 or (not 12>=3 and 4==3)
```

First look at the logic to determine whether each expression is `True` or `False`. Then copy the expressions into a program and use `print(..)` with the logical expression in brackets to see if you had the correct answers.

---

(29) ☐ Write a program to input a number from the keyboard, *n*, and then compare it to a number you define in the program, *m*. Your program should return one of the following three messages

- You entered *n* which is less than *m*
- You entered *n* which is equal to *m*
- You entered *n* which is more than *m*

Test this program where you define integer values for *m*. Test also with floating point numbers for *m*.

Note that there can be an issue comparing two floating point numbers to see if they are equal to each other.

---

(30) ☐ Consider the quadratic equation

$$ax^2 + bx + c = 0$$

Write a program to input numbers from the keyboard for the constants a, b and c in this quadratic equation.

Your program should return one of the following messages about the number of roots that the equation has;

- There are two real roots of this quadratic equation.
- There is one real root of this quadratic equation.
- There are two complex roots of this quadratic equation.

Try different values for a, b and c.

Once you have the program working then try the following values for a, b and c.

a	b	c	a	b	c
2	7	3	0.04	0.1	0.25
1	2	10	0.25	-0.2	0.08
4	-12	9	0.1	0.29	0.1

Check to see if you program gave the correct answers.

Note that for some vaues of a, b and c you should read the values in as integers, with `int()`, and for some values you shoud read them in as floating point numbers, with `float()`.

Hints:

- Remember that the equation for solving a quadratic equation is

$$x = \frac{-b}{2a} \pm \frac{\sqrt{b^2 - 4ac}}{2a}$$

- Note from this equation that the value of $(b^2 - 4ac)$ is critical to figuring out how many roots there are and what type of roots they are; whether they are real or complex.
- There is a special library for complex numbers, `cmath`, but you do not need to use this library here. `cmath` allows calculations to be made using complex numbers, but the calculation of $(b^2 - 4ac)$ is a real number calcuation - and this is all you need to determine whether there are one or two roots and whether they are complex or real.

---

(31) ☐ Modify your program from exercise 30 so that it prints out the roots of the equations as well as the message to say how many roots there are.

Hints:

- Again for this exercise, you do not need to use `cmath`, the library for complex numbers.
- To calculate a complex root then you use different parts of equation **??** to determine the real part of the root and the imaginary part of the root. Again the real part and imaginary part of the root can be calculated with real number calculations.
- To display a complex root where the magnitude of the real part is equal to `x` and the magnitude of the imaginary part is equal to `y` a print statement can be used such as;
  ```
  print(x,"+",y,"j")
  ```
- Python has a variable type which is 'complex'. So, for example, the following statments make a complex number and print it;
  ```
  z = complex(x,y)
  print(z)
  ```
  Here, the complex number `z` is made from `x` and `y`, which are again the real and imaginary parts of a complex root; thus, $z = x + \imath y$. Note that this makes the print statement easier and

that `complex(x,y)` and `print(z)` can be used without using the `cmath` library.

4.7. **Loops: while and for.** Loops are very important in programming. In Python there are `while` and `for` loops.

An example of a `while` loop is;

```
i=0
while i<5 :
  print(i)
  i+=1
print("the loop has ended")
```

If the logical expression after the `while`, above `i<5` is true then all the commands that are indented will be carried out. Whenever the logical expression is false the program will continue and ignore the loop commands.

Can you predict which numbes will be printed out?

Copy and paste the code above into a program and try it to see if you were correct.

Note that without the `i+=1` command at the end the loop would go on for ever because the value of `i` would always be lower than 5. The situation where a loop goes on for ever is called an 'infinite loop'.

The other type of loop is a `for` loop;

```
for i in range(5) :
  print(i)
print("the loop has ended")
```

The `range` function generates a set of numbers. The behaviour of `range` depends on the number of arguments it has. Here 'number of arguments' means the number of different numbers in the brackets after it. The behaviour is summarised in the following table;

function	number range	output with a=7, b=3, c=2
range(a)	zero to a-1	0,1,2,3,4,5,6
range(b,a)	b to a-1	3,4,5,6
range(b,a,c)	b to a-1 in steps of c	3,5

Note that the arguments of `range`, the numbers in the brackets, must be integers and cannot be floats.

There are two more important commands for loops; `continue` and `break`. For example,

```
for i in range(5) :
  if i==1 : continue
  print(i)
  if i==3 : break
print("the loop has ended")
```

The output of the program above is

```
0
2
```

```
3
the loop has ended
```

Note that `1` is not printed because when `i=1` the `continue` function restarts the loop with the next value of i *before* the `print` statement. Note also that the loop finishes earby because when `i=3` the `break` command finishes the loop early. Note finally that `3` is printed because the `break` command is after the the `print` statement in the loop.

4.8. **exercises.**

(32) ☐ Print out the numbers `0` to `9` inclusive on the screen.
Try doing this exercise using `while` loop and then try again with a `for` loop.

---

(33) ☐ Modify your loop program to print out the numbers `0` to `18` in steps of `2` on the screen; 0, 2, 4, 6, 8, ... 18.
Again try doing this exercise using `while` loop and then try again with a `for` loop.

---

(34) ☐ Modify your loop program to print out the numbers `12` to `-6` in descending steps of `3` on the screen; 12, 9, 6, 3, 0, -3, -6.

---

(35) ☐ Print out all the integers betwee `1` to `30` inclusive on the screen. (1, 2, 3,... 29, 30)

---

(36) ☐ Print out the numbers `1` to `30` inclusive on the screen except for numbers which are multiples of `3`. You will need to use a loop and an `if` decision statement inside the loop.

---

(37) ☐ Repeat exercise 36 except do not print numbers that are multiples of either `3` or `5`.

---

(38) ☐ input an integer value `n1` and then display a row of `n1` asterisk symbols `*`.
Note `print("*",end="")` will print out one star *without printing a newline afterwards*. If you put this in a loop you can print it `n1` times. Sometimes there are 'buffering' problems and things do not print without the newline because they get stuck in a 'buffer'. If this happes it is necessary to flush the buffer with

```
print("*",end="",flush=True)
```

Another useful thing that can be put into a print command is `sep=""`, which stops a white space character being printed between different items. Of course, what `end=""` and `sep=""` are doing is defining what goes at the end of a print and what goes in between two different items; by default these are a newline and a single white space respectively.

---

(39) ☐ Use a loop to produce the following output on the screen, or something similar to it.

```
0  0.0
1  0.1
2  0.2
3  0.3
4  0.4
5  0.5
6  0.6
7  0.7
8  0.8
9  0.9
10  1.0
11  1.1
12  1.2
13  1.3
14  1.4
15  1.5
16  1.6
17  1.7
18  1.8
19  1.9
20  2.0
```

---

(40) ☐ The aim here is to generate some data with the function

$$y = \frac{x}{1 + x^2}$$

by modifying the program from exercise 39 so that you print to the screen the following output of x and y values

```
0.0  0.0
0.1  0.099
0.2  0.192
...  .....
...  .....
2.0  0.400
```

(41) ☐ *(optional)* Repeat exercise 40, but input from the keyboard the start value for x, the end value for x and the stepsize between x values.

(42) ☐ **f-strings** since Python 3.6 a new way of formatting strings has been introduced, which can be useful with `print`. For example;

```
a=12;b=3.141;name="computer"
print(f"{a} hots dogs with {b} g of sauce for {name}")
# prints.. 12 hots dogs with 3.141 g of sauce for computer
```

Reapeat exercise 40 using an 'f-string' inside the `print` statement.

For more information about f-strings see this article and/or python documentation.

4.9. **Functions.** It is really helpful to be able to write your own functions in a program. First, functions are introduced and then their benefits are considered.

An example of a fuction is `factorial` which is sent a single parameter, `n`. Remember that the factorial of an integer, n!, is the product of all the integers from 1 to n. For example, 3! = $1 \times 2 \times 3 = 6$.

```
def factorial(n):
  b=1
  for c in range(2,n+1):
    b*=c
  return b
```

which needs to be put at the top of the program before you call it.

The function returns the factorial of the number sent to it. Notice the indentation. Inside the function is a `for` loop where b is multiplied by all the integers from 2 to n. The `return` statement sends the calculated factorial, b back.

Note that the code to call the function could be something like

```
answer= factorial(4)
# or
w=7
answer=factorial(w)
```

and note that it is not necessary to have the same variable name in function itself and the place where the function is called. It is only necessary for the argument sent to be of the correct type. So, for example, the function `factorial` above requires an integer, because `range` requires an integer.

Functions don't have to return a value. For example, the function `hello` doesn't take any parameters and doesn't return anything.

```
def hello():
  print("Hello!!")

# can be called with
hello()
```

There are several advantages to using functions. For example,

- They help split the program into small manageable chunks
- Individual functions can be tested to make sure they behave as expected to help produce reliable programs
- They enable code to be reused

4.10. **Exercises.**

(43) ☐ Write a program that uses the `factorial` function given above to calculate the factorials of integers from 1 to 10 and print them to the screen.

---

(44) ☐ Modify your program from 38 ao that it includes a function `asterisk(n)` that prints a row of `n` asterisks to the screen.

---

(45) **Mini-projects in number theory** These mini-projects are all *optional*. They are some ideas for programs you could write to practice programming.

(a) write a program to test to see if an integer is a prime.
(b) write a program to find the first n prime numbers; here a natural strategy is to turn your program from exercise 45a into a function and then call it from within a loop in `main`.
(c) write a program to check if a number is a perfect number. A perfect number is a number that is equal to the sum of its positive factors including `1`, but excluding the number itself. For example, `6` is the smallest perfect number as it has positive factors `1`, `2` and `3`.
(d) Extend your program from 45c to find the first n perfect numbers. Note that if $n > 4$ then the program will take a long time. The program can also be modified to search for near perfect numbers where the sum of factors is equal to the number plus or minus a small integer.
(e) modify your perfect number program from 45d to find the first n 'near-miss' perfect numbers where the sum of factors is equal to $n + \delta$ or $n - \delta$ where δ is a small integer.

---

5. PYTHON TUTORIAL
LISTS, TUPLES, ARRAYS AND DATA HANDLING

(return to contents)

In many programming languages 'arrays' are very important way of storing information. Before we start to look at how arrays in Python it is important to think about what arrays are and why they are useful.

Arrays are normally used to store many pieces of related information. So far all the variables we have used have had a single piece of information, such as a single number. An array is designed to hold multiple pieces of information in a convenient way.

It can be helpful to think about how an array works. Imagine that in an experiment you made 180 different measurements of temperature over a three hour period. You wouldn't want to have 180 different variable names in a program to store each temperature individually. You can have a single variable name, *e.g.* `temperature`, with many, *e.g.* 180, different values. To access each different value you use an 'index'.

In this example of temperature measurements;

```
temperature         # refers to all the information
temperature[0]      # this is the first temperature measurement
temperature[1]      # this is the second temperature measurement
temperature[n-1]    # this is the nth temperature measurement
temperature[n]      # this is the (n+1)th temperature measurement
temperature[179]    # this is the 180th and last temp. measurement
```

So now if we remember that how the `range()` function works then `range(180)` gives numbers from 0 to 179 inclusive so consider the code below

```
sum=0                     # needs to have initial value of zero
for n in range(180):
  sum+=temperature[n]     # adds up all the measurements of temp.
average_temperature = sum / 180
```

This code is a simple example of how using arrays and loops we can make useful calculations on large data sets with only a few instructions.

Now, in Python it is actually easier because there are two functions; `sum()` and `len()` which calculate the sum of a list of numbers and count the number of numbers in the list. So the code above can be replaced with

```
average_temperature = sum(temperature)/len(temperature)
```

5.1. **Lists, tuples and arrays in Python.** Two types of data structure that are similar to arrays in Python are the 'tuple' and 'list'. They can be defined in a program with

```
example_list=[10,3,8,4]
example_tuple=(10,3,8,4)
example_empty_list=[]    # we can add members to a list
example_empty_list.append(5)    # this adds a 5 to the list
                         # (we cannot add members to a tuple)
mixed_list=[3,9.34,"robot",3+j]
```

The big difference between a list and a tuple is that a list is 'mutable' and a tuple is 'immutable'. Mutable means a list can be changed and immutable means that a tuple cannot be changed. You can add members to a list with `append`. You can change the values of items in a list. You cannot add members to a tuple. You cannot change the values of items in a tuple.

Lists and tuples are versatile because they can have different types of member. For example, the `mixed_list` above has a string, an integer, a float and a complex number.

What about arrays? Arrays don't and do exist in Python. They don't exist in Python natively, but they do exist because they can be added by adding libraries. Part of the reason that Python is so popular and successful is because there are many excellent libraries which make Python very versatile. We will be thinking about libraries later. One to mention here though is 'numpy', which has arrays and lots of functions that are useful for handling numbers and data sets.

5.2. **Exercises.** In the exercises below we are going to be working with lists in Python.

(46) ☐ Write a program where you make a list with 10 integers from 0 to 9 inclusive using a loop. You will need to start by defining an empty list, such as `nums`. Then if you use `range()` in a `for` loop you can generate the number from 0 to 9 and use `.append()` to add each number in turn to the list.

To check that your program is working correctly you can use the command `print(nums)` to print your list after you have appended all the number to it. You could also put this command in the loop to watch the numbers being appended.

---

(47) ☐ Now modify your program from exercise 46 so that you assign a value of `n*(n+1)/2` to the member [n]. So for the first member of list, `n=0` and `n*(n+1)/2=0` of the list and print out the results to the screen as below.

```
0  0
1  1
2  3
```

`etc.`

Note that to print the list in this way you can use another loop and `print(n,nums[n])`.

Do you recognize these numbers?

---

(48) **Mini-projects in data processing** These mini-projects are all *optional*. They are some ideas for programs you could write to practice programming.

For these exercises you need to have access to a set of data.

To read in a list of numbers from a text file and put it in a list the following code can be used.

```
with open("number_list_2.txt","r") as fin:
  text_list=fin.readlines()
  #  read in every line of data into a text list
number_list = []
for text_item in text_list:
   number_list.append(float(text_item))
print(number_list)
```

Note the use of `for text_item in text_list` means perform a loop where `text_item` will loop through each member of `text_list`.

Below the text will assume that `dat` is name of your list of data.

(a) Find the minimum value for `dat` and the maximum value for `dat`If possible look through the data file to check that you have the identified the correctmaximum and minimum values of `dat`.

(b) Find the mean (average) value of `dat`. Remember that the mean average is the sum of all values divided by the number of values.

(c) The aim of this exercise is to find the mode; the mode average is the most common value in `dat`. It has to be done carefully. You should set a minimum value, a maximum value and a step size, such as 10, 40 and 5 and then collect data for a histogram by finding the number of `dat` values in each interval. So with the example of 10, 40 and 5 you should find the number of `dat` values in the following ranges;

$$10-15 \qquad 15-20 \qquad 20-25 \qquad 25-30 \qquad 30-35 \qquad 35-40$$

When you have found the number in each range you should print the results to the screen.

Hints:

- set up a list where all the list members are zero. Each different member of the list is going to be the number of `dat` values in a particular range, or we might say in a particular 'bin'.
- loop through all the values of `dat` and work out which 'bin' each member of the list should go in. Then add one to

the number of items in that bin. Remember that you are recording the number of items in each bin with another list suggested in the hint above.

---

(49) **Mini-projects in data sorting** These mini-projects are all *optional*. They are some ideas for programs you could write to practice programming.

For these exercises you need to have access to a set of data.

The aim here is to sort a list of data, which will, again, be called `dat` here. When the list is sorted it will be straightforward to find the median average value. The median is the value of `dat`, for example, where half of the `dat` values are higher and half of the `dat` values are lower. An approximate value of the median could be found by modifying the histogram in Exercise 48c above to show cumulative frequency, but for this exercise the data should be sorted from the smallest to the largest value of `dat` to find the median.

List sorting is a classic problem in programming. We take it for granted that Excel, for example, can sort a list of numbers from highest to lowest or lowest to highest. Here we will look at two routines to do this; Bubblesort and Quicksort.

(a) **Bubblesort**. In Bubblesort the computer looks down the list and compares each adjacent pair of numbers making swaps between numbers. After looking down the list once the computer goes down the list again from top to bottom in the same way. It keeps on going through the list until it goes down the list and does not have to swap any values.

So, for example, if we want a list from smallest `dat` at the top to largest `dat` at the bottom the algorithm can be summarized by the following steps;

- If the first number at the top is greater than the second number next to it then swap these two number. If the numbers were swapped then the second number was previously the first number. If the numbers were not swapped then the second number is the same as it was at the beginning. It is important to have the correct value for the second number as the second number is used in the next step.
- Compare the second and third numbers and swap if the second is greater than the third.
- Continue comparing adjacent numbers going down the list swapping if the number closer to the beginning of the list is greater than the number which comes after it.

- repeat the sorting routine from the top to the botton of the list until there is a pass down the list where no numbers are swapped. When no swaps are required the list is sorted.
- Note that at the end of the first pass the largest number must be at the bottom of the list and so in the second pass through the list it is not necessary to do the comparison between the last and last but one elements. Thus, after each pass through the list one less pair of numbers need to be compared.

(b) **Quicksort I**. The time taken by Bubblesort is proportional to n^2 on average where n is the number of items in the list. Quicksort is much more efficient; the time is on average proportional to $n \log n$. Quicksort is a recursive algorithm and before tackling it there are some short recursion exercises.
Recursion - in recursion a function calls itself. So, for example, to find the factorial by recursion

```
def factorial(n) :
  if n==1 :
    return 1
  else :
    return n*factorial(n-1)
```

- **Put this routine in a program and test its behaviour.**
- **Modify this routine to return the sum of 1, 2, 3,... n.**
- **modify the factorial routine to return a fibonacci number f(n)** where

$$f(0) = 0 \qquad f(1) = 1 \qquad f(n) = f(n-1) + f(n-2)$$

(The Fibonnacci numbers are 0,1,1,2,3,5,8... etc.)

(c) **Quicksort II**. In the quicksort algorithm the first number on the list is chosen as a 'pivot' and all the numbers in the list are tested against it. When the function is finished it should have a list of numbers smaller than the pivot number above the pivot number and a list of numbers that are larger than the pivot below it. The last thing the function will do is to call itself two times more for the list of numbers above the pivot and for the list of numbers below the pivot.
The logic of the quicksort function is shown below. The function receives a list of numbers and the length of the list. two counters are used; n, which starts at 0 and m, which starts at (length of the list)-1. So for a list of 5 numbers n starts at 0 and m starts at 4. The logic of the sorting routine is described in the following 'pseudo-'code.

```
**loop point**
-if n+1 > m then exit loop
```

```
-if (number n+1) < (number n) then swap number n and number
 n+1 and increase n by 1 and return to loop point,.. else..
-.. else swap number n+1 and number m and decrease m by 1
 and return to loop point
**end of loop**
-if n > 1 sort the numbers from 0 to n-1 by calling the
quicksort function
-if n +1 < (length of list-1) sort the numbers from n+1 to
 (length of list -1) by calling the quicksort function
-return
```

6. PYTHON TUTORIAL - LIBRARIES

(return to contents)

There are lots of useful libraries that can provide useful functions in Python. For example;

- `random` has pseudo-random number routines.
- `numpy` has many useful numerical functions and arrays.
- `matplotlib` has useful data plotting functions.

To import libraries into your Python program so that their functions can be used the `import` command can be used;

```
import random
# to get a random integer between 0 and 9 inclusive
ran = random.randrange(10)
# (the arguments for randrange() are the same as for range() )
# or
ran = random.randint(0,9)
# ranint(a,b) is equivalent to randrange(a,b+1)
# ranint(a,b) returns integer where a<= int <= b
#
import numpy
# we call numpy functions with numpy.
area = numpy.pi * radius **2
# numpy.pi gives value of pi =3.141....
```

Note that `\numpy` can also be imported with

```
import numpy as np
# now we call numpy functions with np.
area = np.pi * radius **2
# np.pi gives value of pi =3.141....
```

Finally, we can import 'sublibraries' with, for example,

```
import matplotlib.pyplot as plt
#  pyplot provides functions that make matplotlib similar to matlab
# (or an equivalent import statement is...)
from matplotlib import pyplot as plt
#
# if we set up data;
x = [0,1,2,3,4,5,6]
y= [1,1,2,3,5,8,13]
# we plot with
plt.plot(x,y)
#  and produce a graph as a file with
plt.savefig("xyplot.png")
#  to print a plot on the screen use
plt.show()
```

6.1. **Types of library.** There are different types of library in Python. For example, `random` is part of the 'standard library' (see docs.python.org/3/library/). Other libraries, such as numpy and matplotlib are widely available and widely used, but not part of the set of standard libraries.

There is potential for confusion. For example, the `numpy` library has a 'sublibrary' called `random`, or `numpy.random`, which is different to the `random` standard library. It is appropriate here to mention another library, `secrets`, which generates cryptographically secure random number sequences.

6.2. **exercises.**

(50) ☐ **Practice using matplotlib.pyplot** There is an example of using `matplotlib.pyplot` above to produce a simple graph. Take that code and modify it to plot graphs of $f(x) = x^2$ and $f(x) = x^3$ in the range $-10 < x < 10$. You could also plot $f(x) = \sqrt{x}$ in the range $0 < x < 10$.

XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXx

---

(51) ☐ **Guess the number game**. Write a program where the computer generates a random number and the user has to guess it. The user should be told if the random number is higher or lower than the guess after each guess. The user should continue guessing until they hit the correct value of the random number. At the end the user should be told how many guesses it took. You could add comments on how the user did like, for example, 'hole in one' if the user gets the number on the first guess.

Hint Import `random` as described above and use

```
random.randrange(256)
```

to generate a number between 0 and 255 inclusive. Input guesses from the keyboard in a `while` loop that continues until the guess from the keyboard is equal to the random number. At the end the number of guesses required to find the random number should be presented.

Might be interesting to try different ranges for the random number.

---

(52) ☐ **Word search/anagram game**. Write a program that reads in a list of words from a file and puts them in a list. One of the words should be selected at random and then the order of its letters should be scrambled to make an anagram[1]. The challenge to the user is to figure out what the word is.

Note that a file, `word_list_2.txt` is provided for this exercise. This list has many commony found words, but all the words are at least five letters long.

[1]Well, technically not an anagram because normally an anagram makes sense. For example, an anagram of 'silent' is 'listen', but 'tselin' is not normally considered to be anagram of 'silent'

Hints Import `random` from `numpy` again and the expression

```
word=random.choice(word_list)
```

will be helpful. Now `word` will be a string and if you print it you will find a `\n` or 'newline' character at the end of the word. To remove this character the function `strip()` can be used;

```
word=word.strip()
```

`word` is a string and strings are immutable, which means they cannot be changed.Thus, we cannot swap letters around in a string; to be able to swap letters around we need to convert it to a list of individual characters. A string can be converted to a list with the command `list()`, as in

```
character_list=list(word)
```

which is a 'type-conversion'. The individual characters in the list can be accessed as normal with `character_list[a]` for example, where `a` is an integer. Now two characters can be swapped with a piece of code like

```
char=character_list[a]
character_list[a]=character_list[b]
character_list[b]=char
```

but remember that you will want to randomly select the indexes `a` and `b`. Remember that we can get the number of characters in the list using the `len()` function. Furthermore, you will want to make several swaps to scramble all the letters. Therefore, it would be a good idea to put the swapping routine into a function. Finally we need to swap the list back into a string to be able to print it out on the screen. A useful function for this is `join()`; for example,

```
output="".join(character_list)
```

The `""` at the beginning is the string that is put in between each member of the list inside the brackets. In this situation we just want to put all the letters back together so we do not want anything between list members and we use `""`, an empty string, as the separator.

---

(53) ☐ **A simple spell checker**. A simple spell checker can be made by loading words from a file to make a list. For this exercise the file, `word_list.txt` is provided; it has many commony found words and, unlike `word_list_2.txt`, it includes words with less than five letters.

The file should be read into a list. Again there will be an issue with `\n` characters at the end of each string in the list and again `strip()` can be used to remove them, for example, with

```
word_list[a]=word_list[a].strip
```

which could be part of a loop where the index `a` runs from 0 to the length of the list minus one, which are numbers `in range(len(word_list))`

Once the list of words is generated a string can be entered from the keyboard, which is the word to check. In Python `in` is a handy tool for checking if a certain object is part of a list; for example,

```
if input_word in word_list:
```

This expression tells Python to search the `word_list` to see if it contains `input_word`. Note that `in` looks for an *exact* match, which is why the `\n` characters had to be removed from the strings.

Finally, optionally, you could add an option to 'add' a word that is not found in the list to the list. Remember the `append()` function to add a new member to a list. If the list was modified and you wanted to use it again then you woud need to save the updated list after it had been modified.

---

(54) ☐ **Square root function** The aim of this exercise is to see how a library function might be made.

The challenge here is to write a function which uses only +, -, *, / to calculate square roots. The method or algorithm is as follows;

- function sqroot receives `Num`
- first approximation, `Test`, is assigned. This can be 1.
- next approximation is calculated with

  ```
  Test = (Test + Num/Test)/2
  ```

- If `(Num - Test*Test)` is not zero repeat the previous step
- return vallue of `Test`

Note that you should write a loop for this function and it is helpful to print out values of `Test` and `(Num - Test*Test)` each time you go around the loop to make sure it is behaving the way you expect.

Note also that you should think carefully about how you decide if the loop should end when you test the value of `(Num - Test*Test)`.

Finally, you might want to think about what value the initial test should be.

---

7. Introduction to cellular automata and Numpy arrays

(return to contents)

In cellular automata time moves forward in discrete steps and variables are not continuous, but have discrete values. For example, a system may consist of a set of individual cells that can each contain a value of 1 or 0. The most famous example of a cellular automaton is probably the 'Game of Life' of John Conway.

7.1. **Arrays using the Numpy library.**

Here is a good point to introduce arrays from the Numpy library. We saw in section 5 that Python does not have native arrays; instead it has lists and tuples. We can, however, use arrays in Python if we import the `numpy` library. The code examples below import `numpy` as `np` so wherever you see `np.function` it is equivalent to `numpy.function` as described in section 6.

7.1.1. *Creating arrays.*

Arrays can be created with all the numbers in them specified. For example,

```
import numpy as np
array1 = np.array([1,1,2,3,5])
array2 = np.array([[1,2],[3,4]])
print (array1)  ## output is [1 1 2 3 5]
print (array2)  ## output is [[1 2]
                ##            [3 4]]
```

generates two arrays; `array1` is a one dimensional array and `array2` is a two dimensional arrays. The initial values of each member or element of `array1` and `array2` are set to the values in square brackets when the array is created. The 'shape' of the arrays can be see by printing out the arrays - the output is shown in the comment after the print statement.

Individual members or elements of the `array1` can be accessed like list ot tuple memebers with `array1[0]`. `array1[1]` and so forth. For the two-dimensional `array2`, which is a 2×2 array, we need to use two 'indexes' to identify each element; thus,

```
import numpy as np
array2 = np.array([[1,2],[3,4]])
print (array2[0][0]) ## output is 1
print (array2[0][1]) ##           2
print (array2[1][0]) ##           3
print (array2[1][1]) ##           4
```

Arrays can also be defined with initial values of zero, one and anything you would like by using the following functions;

```
import numpy as np
array3 = np.zeros(5)
array4 = np.ones((2,3))
array5 = np.full(5,0)
array6 = np.full((3,3),3.)
print (array3) ## output is [0. 0. 0. 0. 0.]
print (array4) ## output is [[1. 1. 1.]
               ##            [1. 1. 1.]]
print (array5) ## output is [0 0 0 0 0]
print (array6) ## output is [[3. 3. 3.]
               ##            [3. 3. 3.]
               ##            [3. 3. 3.]]
```

where `numpy.zeros` sets all the elements to zero, `numpy.ones` sets them all to one and `numpy.full` fills the array with the number specified.

Note that there is an important difference between the numbers with and without the decimal point marker after them, which can be see in the following code;

```
print (type(array3[0]) ## output is <class 'numpy.float64'>
print (type(array5[0]) ##           <class 'numpy.int64'>
```

So note that the decimal point indicates that the number is a floating point number, but without a decimal point the number is an integer. Thus, the `numpy.zeros` and `numpy.ones` functions generate arrays of floating point numbers, but we can generate an array of integers with `numpy.full` if an integer is specified as the number to fill the array with.

7.1.2. *Calculations with Numpy arrays.* There are some easy ways to perform calculations with `numpy` arrays. Some of them are illustrate below in print statements with the output shown as comments after the print;

```
import numpy as np
array1 = np.array([1,1,2,3,5])
array5 = np.full(5,0)
print (array1+3) ## output is [4 4 5 6 8]
print (array5-2) ## output is [-2 -2 -2 -2 -2]
print (array1*3) ##           [ 3  3  6  9 15]
print (array5-array1) ##      [-1 -1 -2 -3 -5]
print (array5*array1) ##      [0 0 0 0 0]
print (np.mean(array1))  ##    2.4
print (np.median(array1))  ##  2.0
```

Thus, it is possible to add, subtract, multiply and divide every element of a `numpy` array by a constant in a single command. Similarly each element of two arrays *with the same shape* can be added, subtracted, multiplied and

divided by each other. Finally, the mean and median of the numbers in the array can be found with `np.mean` and `np.median`.

7.2. **A one dimensional cellular automaton.** The aim is to develop a computer program to simulate a one dimensional cellular automaton, which consists of a row of 'cells' in a line.

The computer program needs the following components:

- A list or array where the contents of each cell are stored. Each cell will contain either a zero or a one.
- A list or array to store the new state of the cellular automaton, which is calculated from the previous state.
- a function to print out the current state of the system
- a function to calculate the new state of the system from the old state of the system
- a loop to go through a fixed number of steps

The width of the output window in repl.it depends on the width of the browser window. It is suggested that you make the array or list 50 cells wide assuming that you can print 50 characters across the output window. Start with all the cells being equal to zero except for a single 1 in the middle.

When you print out the state of the system you may want to make sure you don't print any spaces between the elements of the array and things look better if you print '#' instead of 1 and ' ' (a space) instead of 0.

A simple rule for calculating the next state of the system to get the program working is that a cell in the new state will have a 1 if in the previous state either the cell to the left, or the cell to the right, or the same cell contained a 1. With this rule you should get a solid triangle printed. It turns out that this rule is called rule 254.

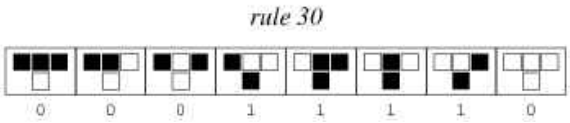


FIGURE 1. Rule 30

7.2.1. *New rules.* Figure 1 illustrates rule 30. It should be clear from figure 1 that there are eight different possible states of three cells which can either have 1 or 0 in them. Now, if we consider these three cells to be the cell to the left of the present cell, the cell itself and the cell to the right of it there are eight different possibilities for how the '1's and '0's could be arranged in these three cells. Each of these eight initial conditions can either give a 1 or 0 in the new state of the system. If we line up these eight possible results as shown in the figure we get an eight digit binary number, such as 00011110 in figure 1, which corresponds to the decimal number 30. In all there are 256 different rules possible using the system from 0 to 255.

Interesting rules to investigate include 30, 86, 90 and 110.

7.2.2. *New initial conditions.* It is interesting to see what happens if instead of starting with a single cell set to one in the initial configuration fo the cells two or more are set to one. This is worth investigating particularly for rule 120, which is more interesting if intially two or more adjacent cells contain one.

It is also interesting to try the rules 30, 86, 90 and 110 with two cells set to one which are separated a little from each other.

7.3. **The game of life.** The game of life was originally described by Martin Gardner in *Scientific American* **223** (October 1970) pages 120-123 and attributed to John Conway.

The game is played on a 2×2 grid. Each square, or cell, contains a 1 or 0, which indicates life or death. Given that each square has eight neighbours the following rules apply:

- Any dead cell with exactly 3 living neighbours springs back to life.
- Any living cell with less than 2 neighbours or more than 3 neighbours dies.

This game, which has been described as a 'zero-player' game; once the initial conditions are set it moves forward automatically. The size of the grid that this game should be played on is up to you.

One useful piece of code that is not standard Python, but works in `repl.it` and in many other implementations of Python is the following;

```
print("\033[1;1H")
```

This code will generally move the cursor back to the top left corner of the screen. This will help you to write the new state of the system over the previous state.

8. PANDAS DATA LIBRARY

(return to contents)

The Pandas library can be used to make 'dataframes', which are a bit like tables of data. The data in Pandas tables can be saved into Excel files (.xlsx) and Excel sheets can be imported into Excel to make Pandas dataframes.

```
import pandas as pd
import matplotlib.pyplot as plt

# %matplotlib QT    #separate windows
# %matplotlib inline   #in Spyder

dataframe=pd.read_excel("excel_sheet.xlsx")
dataframe["a"]=dataframe["z"]*dataframe["y"]

print(dataframe)

plt.figure(figsize=(10,7))
plt.plot(dataframe["x"],dataframe["y"],".",label="y")
plt.plot(dataframe["x"],dataframe["z"],".",label="z")
plt.plot(dataframe["x"],dataframe["a"],".",label="a")
plt.legend()
plt.savefig("test.png")
plt.show()

dataframe["a"]=dataframe["z"]*dataframe["y"]

dataframe.to_excel("excel_out.xlsx")
```

to run the code above it is necessary to download `excel_sheet.xlsx`. Try to run the code, look at the input files and output files and try to figure out what each part of the code does.

Pandas can also be used to load data from and save data to `.csv` files. Saving to Excel files it is also possible to save workbooks with multiple different sheets.

It is suggested that you try using pandas and look on the web to see what it can do. Individual columns of data from dataframes can be converted to Numpy arrays and Numpy arrays can be converted to columns in dataframes.

9. Numerical solution of ordinary differential equations Part I

(return to contents)

Here we take some first steps in solving differential equations numerically.

9.1. **introduction.** There are some equations in mathematics where it is possible to find an exact answer, but there are other equations where it is not possible. For example, there is an exact solution to the question 'How does a mass attached to an ideal spring move?'. In this case the exact solution is an equation, which can predict the position and velocity (speed) or the mass at any time. It can be proven mathematically that the exact solution is correct.

There are many mathematical problems, however, where it is not possible to find an exact solution. If it is not possible to find an exact solution then normally it is possible to find an approximate answer by making many calculations. We generally call this type of solution where many calculations are made a 'numerical solution'.

In this part of the course we are going to be thinking about numerical solutions to problems. In particular, the problems that will be considered here are problems which are defined by differential equations, which can be used to calculate the rate of change of quantities with time. For example, if an apple falls due to gravity then the rate of change of its position and the rate of change of its velocity (or speed) can be calculated with differential equations.

AIMS - Numerical solution of oridnary differential equations
The aims of this section are to

- Learn different techniques for solving ordinary differential equations numerically
- Evaluate the performance of numerical techniques by comparing their results with analytical solutions for systems with analytical solutions
- Consider methods of confirming the results of numerical solutions when there are no analytical solutions to compare with.

9.2. **Radioactive decay.** The aim of this exercise is to simulate radioactive decay. It is the simplest system we can start with. An advantage of studying radioactive decay is that there is an exact solution so that the reliability of any numerical solution calculated can be tested by comparing the exact solution, which is also called the analytical solution.

In radioactive decay atoms decay from one state to another. Strictly speaking it is the nucleus of the atom that decays. The rate of radioactive

decay is given by the differential equation

$$\frac{dN}{dt} = \frac{-N}{\tau} \tag{1}$$

In this equation N is the number of atoms at time t. The greek letter τ, pronounced 'tau' is the 'lifetime' of the atoms. Lifetime is similar to, but not the same as, 'half-life'.

9.2.1. *Exact analytical solution.* **Write a program for radioactive decay** that uses the exact analytical equation to calculate $N(t)$ the number of atoms remaining at time t. The exact equation is given by

$$N(t) \quad = \quad N_0 \, e^{-t/\tau} \tag{2}$$

Note that N_0 is the number of atoms at the start, when time is zero, or $t = 0$.

This equation can be calculated in Python with

```
Nexact = N0 * math.exp(-t / tau)
```

Note, please use `Nexact` as the variable for N calculated with the exact analytical formula so that it is clear that it has been calculated with the exact formula.

Remember you will need to import the `math` library for the `math.exp` exponential function.

You should use the following values for N_0 and τ;

```
N0=1000
tau=20
```

The total time for the simulation t_{max}, `tmax`, and the timestep δt, `dt`, should be

```
tmax = 100
dt = 10
```

and use `t` as t. Note, it is very important that we start with `t=dt` and not zero.

At the end of each loop cycle you should reccord the values of `t` and `Nexact`. Either

- set up empty lists and append the values of `t` and `Nexact` to the end of the lists after each cycle and then plot the points using `matplotlib.pyplot`
- write the values of `t` and `Nexact` to a file so that you can plot the points with Excel (or Gnuplot).

9.2.2. *Euler simulation.* In the Euler Method the state of a system with one or more variables x_i at $t + \delta t$ is calculated from the state of the system at t with

$$x_i(t + \delta t) = x_i(t) + \delta t . \frac{dx_i}{dt} \tag{3}$$

This method is shown graphically in Figure 2.

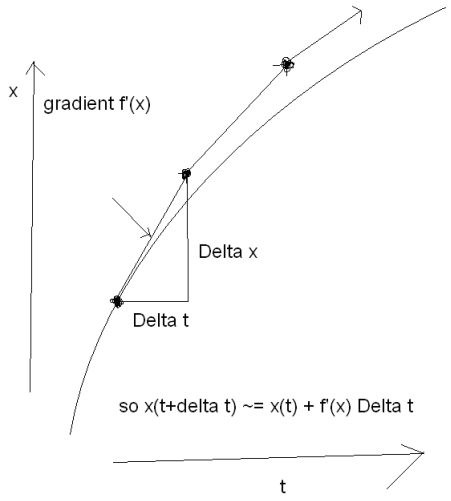


FIGURE 2. Euler Method

The equation for the euler method for radioactive decay is

$$N(t+\delta t) = N(t) + \delta t.\frac{dN}{dt} \tag{4}$$

Now we can calculate $\frac{dN}{dt}$ with Equation (1) above; it is equal to $\frac{-N(t)}{\tau}$.

Thus, in Python the Euler method can be calculated with

```
dNdt = - Neu/tau
Neu = Neu + dt*dNdt
```

where `Neu` is N calulated with the Euler method.

Modify your program for radioactive decay to calculate the N with the Euler method as well as with the exact analytical formula. At the end of the loop over you time you should record `t`, `Nexact` and `Neu` either in lists or in a file so that you can plot the exact solution and the numerical solution calculated with the Euler method on the same graph with `matplotlib` or Excel/Gnuplot.

Investigate the effect of changing the time step δt on the accuracy of the Euler method solution.

Generally, for first tests it may be easiest to just append data to lists and plot with `matplotlib`, but you may want to try saving data as well because when you work on your project it may be helpful to save data files.

10. Introduction to Ordinary Differential Equations

(return to contents)

Before going further with numerical techniques it is necessary to look at ordinary differential equations a little bit more formally.

Generally an ordinary differential equation problem may be defined by a set of first order differential equations of the form;

$$\begin{aligned} \frac{dx_1}{dt} &= f_1(x_1, x_2, ...x_n) \\ \frac{dx_2}{dt} &= f_2(x_1, x_2, ...x_n) \\ &\cdot \quad \cdot \quad \cdot \\ \frac{dx_n}{dt} &= f_1(x_1, x_2, ...x_n) \end{aligned} \tag{5}$$

where x_1, x_2,... and x_n are *independent* variables.

Two variables are independent if they do not depend on each other. For example, in radioactive decay the number of radioactive nuclei, N, is the only independent variable because the rate of change of N, which is of course $\frac{dN}{dt}$, depends on N.

Thus, in radioactive decay the number of radioactive nuclei, N, is equivalent to x_1 in Equation 5 and there is only one first order differential equation because there is only one independent variable. So, from Equation 5 it is necessary to find

$$\frac{dN}{dt} = f(N) \tag{6}$$

Now, the number of radioactive nuclei N drops at a rate proportional to the number of radioactive nuclei so ;

$$\frac{dN}{dt} = \frac{-N}{\tau} \tag{7}$$

where τ is the lifetime of the nuclei.

Another example is Simple Harmonic Motion of a mass, m, attached to a spring, with constant k, where the displacement from equilibrium, x, and velocity, v. Now x and v are both independent variables because if we know x we cannot predict the value of v and similarly if we know v we cannot know the value of x. The accelaration $a = \frac{d^2x}{dt^2}$, however, is a dependent variable because it can calculated from positon, x with the standard equation for a mass on a spring.

$$\frac{d^2x}{dt^2} = -\frac{k}{m}x \tag{8}$$

From Equation 5 it is necessary with two independent variables to have two first order differential equations; one equation for $\frac{dx}{dt}$ and one for $\frac{dv}{dt}$.

$$\frac{dx}{dt} = f(x, v) \tag{9}$$

$$\frac{dv}{dt} = f(x, v) \tag{10}$$

Now it should be clear that $\frac{dx}{dt}$ is equal to the velocity v and because $\frac{dv}{dt}$ is the rate of change of speed or acceleration we can use Equation 8 to get $\frac{dv}{dt}$. Thus we get the two first order differential equations

$$\frac{dx}{dt} = v \tag{11}$$

$$\frac{dv}{dt} = -\frac{k}{m}x \tag{12}$$

Thus, radioactive decay and simple harmonic motion can be represented with a sets of first order differential equations. These two example, of course, have well known analytical solutions. Thus, if we solve these differential equations numerically we can check the answers we obtain against the exact analytical solutions to test to see how accurate the numerical methods are. The analytical solutions are

$$N = N_0\, e^{-t/\tau} \tag{13}$$

$$x = A\, sin(\sqrt{\frac{k}{m}}\, t + \phi) \tag{14}$$

where N_0 is the initial number of radioactive nuclei, A is the amplitude of oscillation and ϕ is a phase factor; all three of these constants depend, of course, on the initial conditions.

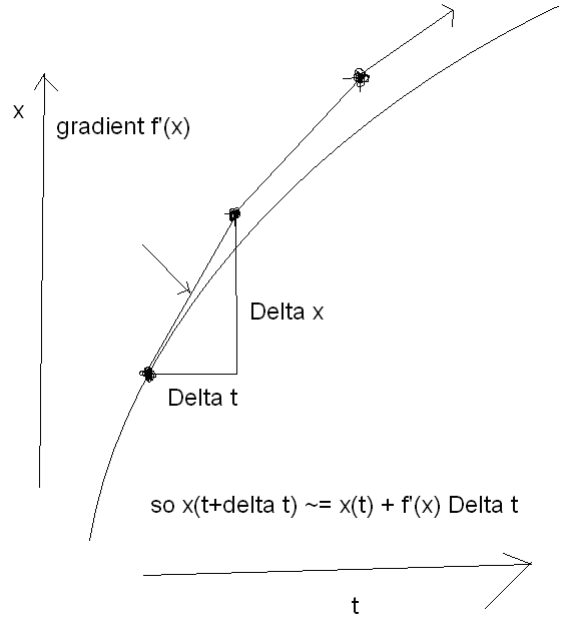


FIGURE 3. Euler Method

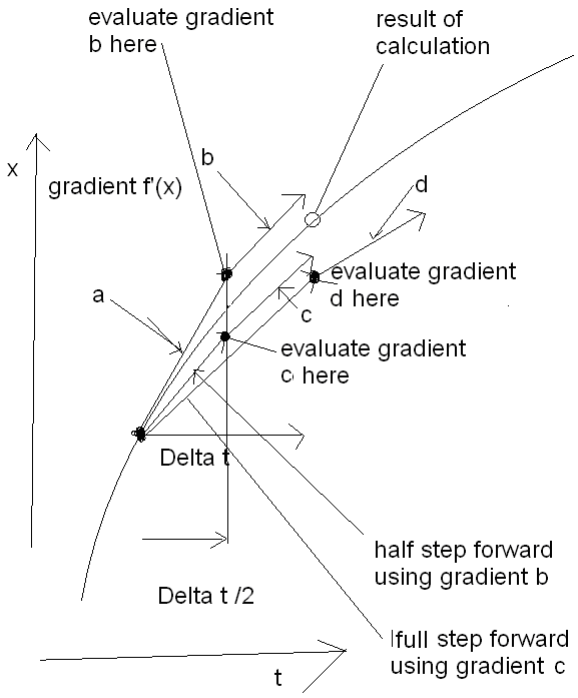


FIGURE 4. Runge-Kutta 4th order method

11. NUMERICAL SOLUTION OF ORDINARY DIFFERENTIAL EQUATIONS PART II

(return to contents)

Here, the Euler, Euler-Cromer and Runge-Kutta methods for solving ordinary differential equations numerically will be considered together.

The Euler and Runge-Kutta (strictly 4th order Runge-Kutta) are represented by Figures 3 and 4. The Euler-Cromer method is less straightforward to describe with a diagram.

As can be seen in Figure 3 the Euler method proceeds by evaluating the gradient at a particular time and then taking a step forward guided by that gradient. By contrast, before taking a step forward the Runge-Kutta method takes several gradients a the starting position and other positions half and full steps forward.

A key part of these numerical methods is the choice of the size of the time step δt

11.1. **Euler Method.**

In the Euler Method the state of a system at $t + \delta t$ is calculated from the state of the system at t with

$$x_i(t + \delta t) = x_i(t) + \delta t \frac{dx_i}{dt} \tag{15}$$

for each different variable x_i. The rate of change with time of each variable, $\frac{dx_i}{dt}$ is determined from the state of the system at time t with

$$\frac{dx_i}{dt} = f_i(x_1, x_2, ...x_n) \tag{16}$$

where x_1 represents $x_1(t)$, and similarly all variables on the right hand side of equation 16 refer to the state of the system at time t.

11.2. **Euler-Cromer Method.**

This is a special case of Euler that turns out to be more stable. It can be applied to systems with two indepent variables, such as x and v in simple harmonic motion as described above.

In general

$$x_1(t + \delta t) = x_1(t) + \delta t \frac{dx_1}{dt} \tag{17}$$

$$x_2(t + \delta t) = x_2(t) + \delta t \frac{dx_2}{dt} \tag{18}$$

where

$$\frac{dx_1}{dt} = f_1(x_1(t), x_2(t)) \tag{19}$$

$$\frac{dx_2}{dt} = f_2(x_1(t + \delta t), x_2(t)) \tag{20}$$

Note that in Equation 20 the value of x_1 in the new position at $t + \delta t$ is used.

11.3. **Runge-Kutta Method.** In the Runge-Kutta method it is more complicated; several different rates of change must be determined. The Runge-Kutta method can be summarized with the equations

$$x_i(t+\delta t) = x_i(t) + \frac{\delta t}{6}(a_i + 2b_i + 2c_i + d_i) \tag{21}$$

for all x_i with $1 \leq i \leq n$ where

$$a_i = f_i(x_1, x_2, ...x_n) \tag{22}$$

$$b_i = f_i(x_1 + \frac{\delta t}{2}.a_1, ...x_n + \frac{\delta t}{2}.a_n) \tag{23}$$

$$c_i = f_i(x_1 + \frac{\delta t}{2}.b_1, ...x_n + \frac{\delta t}{2}.b_n) \tag{24}$$

$$d_i = f_i(x_1 + \delta t.c_1, ...x_n + \delta t.c_n) \tag{25}$$

and again

$$\frac{dx_i}{dt} = f_i(x_1, x_2, ...x_n) \tag{26}$$

Thus, it is necessary in each step to determine four different rates of change for each variable in the Runge-Kutta method, which are labelled above as a_i, b_i, c_i and d_i.

It can be seen from Equations 22 to 25 that it is necessary to calculate these different rates from slightly different states of the system. Here, it makes sense to determine the rates of change in a function.

11.4. **Convergence.** In the two examples of radioactive decay and simple harmonic motion above it is possible to compare the results from numerical solution with exact answers to check for accuracy. It is really important, however, to be able to test the integrity of numerical calculations when an analytical solution is not available.

'Convergence' is a key method for testing the integrity of numerical calculations. In principle, the results of the calculation should not depend on the value of δt, the stepsize. Generally speaking if the stepsize is reduced the calculation should get more accurate. Thus, several calculations can be performed with different values of δt. As δt gets smaller the results of the calculation should 'converge', *viz.* they should approach a limiting value with smaller and smaller differences between successive calculations with smaller δt values. Clearly, if the value of δt has a large effect on the results and convergence is not observed then there is a problem with the calculation; the calculation may be described as 'unstable' in this case. It can be helpful to plot a graph of the final answer as a function of stepsize - or perhaps the log of stepsize to check for convergence.

In the examples of radioactive decay and simple harmonic motion given above, the numerical simulations should 'converge' to the analytical solutions.

11.5. **Symplectic integrators.** Another type of integrator should be considered; symplectic integrators. Symplectic integrators are time reversible, that is to say that the calculation is the same backwards and forwards. So if the integrator is used to go from an initial state to a final state then if the same integrator is run backwards from the final state it should return to exactly the same initial state. This can be a powerful advantage.

Well known symplectic integrators are the Verlet, leapfrog and velocity Verlet methods. Two slightly different forms of the leapfrog method are described below; 'drift kick drift' and 'kick drift kick'. These two integrators work for systems with position and velocity provided that the change in position only depends on the velocity and the change in velocity depends only on the position.

- **drift kick drift**. Move the position forward using the initial velocity with a half step, *viz.* half the time of a normal step. Then using this new position calculate a full step of the velocity. Finally the position is moved forwards by another half step, but using the new velocity determined in the previous step.
- **kick drift kick**. Increase the velocity using the initial position with a half step, *viz.* half the time of a normal step. Then using this new velocity move the position forward a full step. Finally, use the new position to move the velocity by another half step.

Symplectic integrators have the advantage of being very stable for conserving the total energy and angular moment of the system.

11.6. **Adaptive timestep integrators.** The timestep is critical in the performance of integration routines. In some cases the system behaves in way were large timesteps can be taken some of the type, but at other times very small ones need to be taken. Sometimes it is most efficient if possible to 'adapt' the timestep during the simulation, which requires some estimation of th likely numerical error at each step and a decision making process to 'adapt' or change the size ofthe timestep. Such routines are beyone the scope of this guide, but there is plenty of information about them online. One point, however, is that generally speaking the introduction of variable timestep to symplectic integrators breaks the symplecticity.

12. Ordinary Differential Equation - Practical Exercise

(return to contents)

The aim of this exercise is to simulate radioactive decay with the different numerical methods described above and compare the results with exact answers from analytical solutions.

Your goals should be

(1) write a program for radioactive decay that uses the analytical formula to calculate $N(t)$ the number of atoms remaining at time t,
(2) write a Euler simulation of radioactive decay into the same program and output both the analytical and simulated populations at each time step,
(3) find the largest value of δt, the time step, that gives reasonable agreement between the Euler simulation and the analytical formula.
(4) write a Runge-Kutta simulation of radioactive decay and compare this with tha analytical results.
(5) find the largest value of δt, the time step, that gives reasonable agreement between the Runge-Kutta simulation and the analytical formula.

Note that the Euler-Cromer method is not applicable to radioactive decay.

13. Class Assignment 1 - Ordinary Differential Equations

(return to contents)

In this assignment you will determine the maximum stepsize permissable for the Euler, Euler-Cromer and Runge-Kutta methods in the simulation of a simple harmonic oscillator.

13.1. **Set up.** You should model a simple harmonic oscillator with values for k and m determined by your student number; if your number is 12345678 the the value of k will be $56+1 = 57$ and the value of m wil be $78+100 = 178$. Start with a displacement of 0.1 metres and speed = zero.

13.2. **Tasks.**

- Find the maximum timestep, where the values of position and velocity agree with the analytical solution to within 1% of the maximum values of x and v for the first 10 oscillations of a simulation using
  - the Euler method,
  - the Euler-Cromer method and
  - the Runge-Kutta method.
- Comment on your results.
- Write a paragraph about what your program(s) do(es) and print this out with a listing of your program.

13.3. **Write up.** This is meant to be a *short* assignment. The full write up should be a maximum of 1 side of A4 with the your program or programs printed out as extra pages in an appendix.

The write up should be structured in this way;

- Title, name, date, student number.
- A paragraph about what your program(s) do(es) and print this out with a listing of your program.
- Results - a table with the maximum timesteps you found for
  - the Euler method,
  - the Euler-Cromer method and
  - the Runge-Kutta method.
- A paragraph with comments on your results.
- All of the above should be less than one side of A4. On page 2 you should have an appendix with a printout of your program or programs if you used more than one program. The appendix can be longer than one page if necessary.

14. CLASS ASSIGNMENT 2 - PARTIAL DIFFERENTIAL EQUATIONS

(return to contents)

In this assignment you use the numerical partial difference method to solve Laplace's equation to calculate the heat distribution across a simple two dimensional plate. In two dimensions Laplace's equation for the temperature on a two dimensional plate, $T(x, y)$, is

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0 \tag{27}$$

Now if we divide a plate into a set of four 'cells' or points on a grid in two dimensions where the temperature is represented with T_{ij} the model system might look like the following;

	c	c	
d	T_{21}	T_{22}	b
d	T_{11}	T_{12}	b
	a	a	

The cells with a, b, c and d represent boundary conditions at the edge of the plate.

Now the partial differential equations can be represented in the two dimensional grid system with

$$\frac{\partial^2 T}{\partial x^2} \approx \frac{T_{(i-1)j} - 2T_{ij} + T_{(i+1)j}}{\delta x} \tag{28}$$

$$\frac{\partial^2 T}{\partial y^2} \approx \frac{T_{i(j-1)} - 2T_{ij} + T_{i(j+1)}}{\delta y} \tag{29}$$

where δx and δy are the spacing of the cells in x and y. Now provided these spacings are the same the approximations can be put into equation 27 to give

$$T_{ij} = \frac{T_{(i-1)j} + T_{(i+1)j} + T_{i(j-1)} + T_{i(j+1)}}{4} \tag{30}$$

This equation can be used to calculate the values of T_{ij} from the values of a, b, c and d.

14.1. **Set up.**

The values of a, b, c and d are determined by your student number; if your number is 12345678 the value of a will be 5, the value of b will be 6, the value of c will be 7 and the value of d wil be 8. If the last four digits of your student number are all the same then you should use the first four numbers a is the first digit, b the second and so forth.

14.2. **tasks.**

- Find the values of T_{11}, T_{12}, T_{21} and T_{22}.

14.3. **Write up.** This is meant to be a *short* assignment. The full write up should be a maximum of 1 side of A4 with the your program or programs printed out as extra pages in an appendix.

The write up should be structured in this way;

- Title, name, date, student number.
- A paragraph about what your program(s) do(es).
- Results
  - The final values of T_{11}, T_{12}, T_{21} and T_{22}.
- A paragraph with comments on your results.
- All of the above should be less than one side of A4. On page 2 you should have an appendix with a printout of your program or programs if you used more than one program. The appendix can be longer than one page if necessary.

15. PROJECTS

(return to contents)

The most important piece of work that you will do in this part of the module is a computational physics project. This project can be made in a number of different areas. These projects are summarized here, classified by the type of computational problem;

(1) Differential Equations
   - Investigation of damped/driven oscillation including chaotic motion of a damped/driven pendulum.
   - Motion of astronomical bodies in the system including two-body, three-body and n-body problems, and Kirkwood gaps.
   - Shooting methods for solving the Schrödinger Equation in one dimension.

(2) Partial Differential Equations
   - Electric potential calculations including determination of the potential around a dipole.
   - Waves on a string including simulation of the propagation of a wave along a string and standing waves. Could include the use of an FFT - Fast Fourier Transform - to analyse the frequency components of the wave on the string.

(3) Stochastic (Random) Systems
   - Percolation; investigation of open lattices where some sites are filled and some empty
   - Percolation - immunization and disease spread

(4) Other topics
   - Protein folding including simple two dimensional simulations; could include three dimensional folding.
   - Your own choice! This must be discussed prior to approval.

15.1. **Notes for all projects.**

- All projects must be discussed and a work plan approved prior to commencement.
- A useful book to look at is 'Computational Physics' 2nd Ed. by Giordano and Nakanishi. This has shelf mark QC20 GIOR in the library.
- You should do a little bit of research in the areas of the projects before choosing your topic.

15.2. **Project guidelines.** *Please read these carefully*

In your project report you will need to present

- An abstract that summarizes your project and the key results/conclusions.
- An introduction to the area and an overview of the computational problem.
- A description of the method that you used to tackle the problem.
- Your results.
- Discussion of results and conclusions.
- An appendix with a printout of your Python program(s).

Please note that there is a word limit of 6000 words as judged by Word/Foxit PDF reader for the *entire* document including all numbers and the computer program.

15.3. **Word limit.** There is a word limit of 6000 words as judged by Word and Foxit PDF reader for the *entire* document including all numbers and the computer program. A word limit was requested in student feedback. An analysis was made of reports submitted when there was no word limit. It was found that over half the reports were submitted with less than 6000 words (using this measure) previously and that these reports of less than 6000 words included reports of all qualities including a report that got the higest marks awarded amonst all the other reports submitted that year.

Please note that no exception will be made for exceptionally long programs or any other reason. The reason that numbers and your python program must be included in the word count is because that makes it easy to check the PDF file and ensure it is ok.

I am happy to discuss this limit and I expect that we will discuss this word limit in class.

Finally, you do not have to use Word to write your report. LaTeX is an alternative and QUB has some deal with Overleaf I think, where you can use LaTeX in a web environment. The reason that the word limit is to be judged with Word or Foxit Reader is because in tests they give identical number of words and Foxit Reader is free software, which can be used to check your word count if you are not using Word.

15.4. **Project mark scheme.** Each project will be marked with the following approximate weightings.

- Structured part of the project

  [50]

- Unstructured part of the project

  [50]

In the project descriptions below each project has a number of tasks for the structured part of the project. **You need to do all the steps in the structured part of the project**.

In the unstructured part of the project it is an opportunity for you to develop your project and explore new areas beyond the structured part of the project. There are some guidelines with each project for what you might

want to do in the unstructured part of the project, but often there are more suggestions that you will have time for. You may also want to pursue your own ideas about how the project could be developed. **You DO NOT need to try to address every suggestion in the unstructured part of the project**.

There are two general points of unstructured work that are relevant to all or nearly all projects.

- You are encouraged to consider how your code could be optimised to make it run faster. If you can make quantitative measurements to quantify any improvements then that could provide some good evidence for your report.
- You are encouraged to consider convergence. It is helpful where possible to test the validity of results by, for example, repeating a calculation with a step size smaller and maybe then an even smaller step size to check that the results are similar and that they appear to 'converge' to a consistent 'final answer'.

16. PROJECT DETAILS

(return to contents)

16.1. **Investigation of damped/driven oscillation including chaotic motion of a damped/driven pendulum.**

16.1.1. *Physics.* The dynamics of a non-linear pendulum can be modelled using the following differential equation

$$\frac{d^2\theta}{dt^2} = -\frac{g}{l}sin\theta \tag{31}$$

which should be split up into the two equations

$$\frac{d\theta}{dt} = \omega \tag{32}$$

$$\frac{d\omega}{dt} = -\frac{g}{l}sin\theta \tag{33}$$

where ω is the angular velocity.

Note that this splitting of a second order differential equation into two first order differential equation was also used for the simple harmonic oscillator above.

To include damping and an oscillating driving force the equations are modified to

$$\frac{d^2\theta}{dt^2} = -\frac{g}{l}sin\theta - q\omega + Fsin\Omega_D t \tag{34}$$

which splits up into

$$\frac{d\theta}{dt} = \omega \tag{35}$$

$$\frac{d\omega}{dt} = -\frac{g}{l}sin\theta - q\omega + F_D sin\Omega_D t \tag{36}$$

where q is a frictional damping coefficient, F_D is the driving force and Ω_D the angular driving frequency in radians s^{-1}. Strictly, F_D is the maximum angular acceleration due to the periodic driving force.

16.1.2. *Structured work.*

(1) Write a program to model a non-linear pendulum. Use the value of l given by

$$l = 0.5 + \frac{5678}{10000}\ metres \tag{37}$$

where 5678 are the last four digits of your student number.

(2) Use your model to determine the period and frequency of oscillation of your pendulum with amplitudes of oscillation of $5°$, $10°$, $20°$ and $40°$.

(3) Plot a graph of frequency as a function of oscillation amplitude for your pendulum. You should use your model to calculate the frequency for a wide range of amplitudes. *N.B.* it will be necessary for your program to automatically determine the frequency of oscillation.

16.1.3. *Unstructured work.*

(1) Modify your program to include damping and an oscillating driving force.
(2) Use your modified program to explore the different types of behaviour that this damped, driven and non-linear pendulum can exhibit.
(3) Your report should include plots to show examples of different types of behaviour.

16.2. **Motion of astronomical bodies in the system including two-body, three-body and n-body problems, and Kirkwood gaps.** *(return to contents)*

16.2.1. *Physics.* The dynamics of the solar system are dominated by gravitational attraction between the different bodies. For two bodies moving in two dimensions, the differential equations are

$$\frac{d^2x_1}{dt^2} = \frac{1}{m_1}.\frac{Gm_1m_2(x_2-x_1)}{r_{12}^3} \tag{38}$$

$$\frac{d^2y_1}{dt^2} = \frac{1}{m_1}.\frac{Gm_1m_2(y_2-y_1)}{r_{12}^3} \tag{39}$$

$$\frac{d^2x_2}{dt^2} = \frac{1}{m_2}.\frac{Gm_1m_2(x_1-x_2)}{r_{12}^3} \tag{40}$$

$$\frac{d^2y_2}{dt^2} = \frac{1}{m_2}.\frac{Gm_1m_2(y_1-y_2)}{r_{12}^3} \tag{41}$$

Where G is the gravitational constant, m_i is the mass of the ith body, x_i and y_i are the x and y positions of the ith body and r_{ij} is the distance between bodies i and j.

As before, each of these second order differential equations need to be split up into two first order differential equations.

$$\frac{d\,x_1}{dt} = vx_1 \tag{42}$$

$$\frac{d\,vx_1}{dt} = \frac{Gm_2(x_2 - x_1)}{r_{12}^3} \tag{43}$$

$$\frac{d\,y_1}{dt} = vy_1 \tag{44}$$

$$\frac{d\,vy_1}{dt} = \frac{Gm_2(y_2 - y_1)}{r_{12}^3} \tag{45}$$

$$\frac{d\,x_2}{dt} = vx_2 \tag{46}$$

$$\frac{d\,vx_2}{dt} = \frac{Gm_1(x_1 - x_2)}{r_{12}^3} \tag{47}$$

$$\frac{d\,y_2}{dt} = vy_2 \tag{48}$$

$$\frac{d\,vy_2}{dt} = \frac{Gm_1(y_1 - y_2)}{r_{12}^3} \tag{49}$$

This is a complex array of equations, particularly for the Runge-Kutta method as x_i, y_i, vx_i, vy_i will all need to be moved a half-step, for example, before the second and third gradients are measured.

When this is extended to n-bodies it is really essential to use arrays for the different values of x_i, y_i, vx_i and vy_i.

$$\frac{d^2x_i}{dt^2} = \sum_{j=1,\ j\neq i}^{n} \frac{Gm_j(x_j - x_i)}{r_{ij}^3} \tag{50}$$

$$\frac{d^2y_i}{dt^2} = \sum_{j=1,\ j\neq i}^{n} \frac{Gm_j(y_j - y_i)}{r_{ij}^3} \tag{51}$$

which split into

$$\frac{d\,x_i}{dt} = vx_i \tag{52}$$

$$\frac{d\,vx_i}{dt} = \sum_{j=1,\ j\neq i}^{n} \frac{Gm_j(x_j - x_i)}{r_{ij}^3} \tag{53}$$

$$\frac{d\,y_i}{dt} = vy_i \tag{54}$$

$$\frac{d\,vy_i}{dt} = \sum_{j=1,\ j\neq i}^{n} \frac{Gm_j(y_j - y_i)}{r_{ij}^3} \tag{55}$$

When arrays are used for the different x, y, vx and vy values for each 'body' it is possible to use loops to help determine the sums in equations 53 and 55. Loops can also be used to move each body forward.

Start each simulation with planets and asteroids moving in circular orbits about the Sun. For this it is necessary that the velocity vectors of the planets and asteroids are perpendicular to their position vectors, where each position vector is the vector from the Sun to the body. It is also necessary for the magnitude of the velocity, v (or speed), of each body to be initially given by

$$v = \sqrt{\frac{Gm_{Sun}}{r}} \tag{56}$$

where m_{Sun} is the mass of the Sun and r is the distance between the Sun and the body.

The Sun should be placed at the origin and fixed in space with zero velocity and zero acceleration at all times. In reality, of course, the Sun moves and a possible extension in the unstructured part of the project is to allow the Sun to move as described below.

16.2.2. *Units and constants.* Your calculation should use SI units of m, s, kg etc., but when plotting graphs and reporting data in tables the following conversion factors will be useful;

$$1\ AU = 1.495978707 \times 10^{11}\ m \tag{57}$$

$$1\ day = 86400\ s \tag{58}$$

$$1\ year = 365.25\ days \tag{59}$$

where AU is the astronomical unit, which is approximately the mean distance between the Earth and Sun. The masses of the Sun, Earth and Jupiter should be taken as

Body	Mass (kg)
Sun	1.98892×10^{30}
Earth	5.9742×10^{24}
Jupiter	1.8986×10^{27}

16.2.3. *Structured work.*

(1) Write a program to model the two body case for the Sun and Earth. Remember that you should assume that the Sun is fixed in space and does not move.
(2) Use your program to determine the time it takes for the Earth to go around the Sun. You should check this against the known value to check your code is working correctly.
(3) Use your program to determine the time it takes for a planet at distance r from the Sun in a circular orbit to complete one revolution around the Sun where r is given by

$$r = 0.4 + \frac{5678}{25000}\ AU \tag{60}$$

where where 5678 are the last four digits of your student number and AU is astronomical units; assume that $1AU = 1.495978707 \times 10^{11}m$ exactly

16.2.4. *Unstructured work.*

(1) Extend you program to n-bodies and run it to simulate the solar system with all the planets.
(2) Include in your calculation an asteroid of mass 0.95×10^{21} kg, orbiting the Sun. Explore the stability of the asteroid in different orbits, *i.e.* orbits with different radii. Note that it may help to calculate the kinetic energy of the asteroid regularly.
(3) Extend your calculations of solar system dynamics. For example, you may wish to change the mass of the asteroid above or change the masses of the planets.
(4) You could also consider allowing the Sun to move. Note that if you allow the sun to move then you should ensure that the total linear momentum of the solar system is zero at the beginning of the simulation. The simplest way to do this is to calculate the total initial momentum of the planets and give the sun the velocity so that its momentum is equal and opposite to the momentum of the planets.
(5) Your report should include plots to show examples of different types of behaviour.

16.3. **Shooting methods for solving the Schrödinger Equation in one dimension.** *(return to contents)*

16.3.1. *Physics.* The time-independent Schrödinger Equation

$$H\psi = E\psi \tag{61}$$

$$\equiv (\frac{-\hbar^2}{2m}\nabla^2 + V)\psi = E\psi \tag{62}$$

in one dimension reduces to

$$(\frac{-\hbar^2}{2m}\frac{d^2}{dx^2} + V)\psi = E\psi \tag{63}$$

which can be rearranged to give

$$\frac{d^2\,\psi}{dx^2} = \frac{-2m}{\hbar^2}(E - V)\psi \tag{64}$$

This second order differential equation can be split into two first order differential equations by introducing ω, which is the derivative of the wavefunction, ψ, with respect to x; $\omega = d\psi/dx$. This is analogous to the introduction of velocity to split the second order differential equation for Simple Harmonic Motion into two first order differential equations.

Equation 64 can be split into the following two first order differential equations.

$$\frac{d\,\psi}{dx} = \omega \tag{65}$$

$$\frac{d\,\omega}{dx} = \frac{-2m}{\hbar^2}(E-V)\psi \tag{66}$$

where m is the mass of the particle, E is the kinetic energy of the particle and V, or $V(x)$, is the potential energy at x.

The 'shooting method': In the shooting method we start the wavefunction from one side of the potential and 'shoot' it across to the other side. So for the infinite potential well in one dimension we start at $x = 0$ and shoot across to $x = L$ in steps of dx. The method can be broken down into the following steps

- guess an energy, E.
- start the wavefunction at $x = 0$ with $\psi = 0$ and $\omega \neq 0$. Thus the initial value of ω is a constant that you can choose arbitrarily.
- propogate ψ and ω along the x-axis.
- count the number times that the wavefunction value ψ crosses the x-axis by changing from positive to negative or negative to positive. If the number of crossings, n, is less than the quantum number of the state you are calculating for then the energy is too low. If the number of crossings is equal to or greater than this quantum number then the energy is too high.
- use bisection to find the energy of the wavefunction to a reasonable degrree of accuracy. For example, you might want to run bisection until the gap between the maximum and minimum energies is less than the average energy divided by a million or some other high power of ten.
- when you have found the energy you may want to run a final shooting calculation at the final energy and record the values of x and ψ so that you can plot the wavefunction by plotting ψ against x. At this point you may also want to normalise the wavefunction. To normalise the wavefunction you need to find the integral of the wavefunction squared from 0 to L and then divide the wavefunction by the squareroot of the value of the integral.

The method above will work for the infinite one-dimensional potential well with width L, where the wavefunction must be zero at each end of the well. In the case of other wells where the potential is not infinite at the walls the wavefunction will tunnel into the walls. The same procedure can be used for these wells, with the modification that it is necessary to shoot from **inside** the potential barrier. The exact values of x to shoot from and to dependon the potential $V(x)$ and there will be some trial and error. It is suggested that you choose and negative energy, which we will call $-U$, and calculate the

limits of x as the positions at which $E - V(x) = -U$. Note that the limiting x values will change as the trial energy, E, changes.

For the inifinite potential well, the analytical solution for the energy levels, E_n, for a particle of mass, m, in a box of length, L, is given by

$$E_n = \frac{n^2 h^2}{8mL^2} \tag{67}$$

The analytical wavefunctions, ψ_n, are given by

$$\psi_n = \sqrt{\frac{2}{L}} \sin(\frac{n\pi x}{L}) \; 0 < x < L \tag{68}$$

$$\psi_n = 0 \text{ elsewhere} \tag{69}$$

16.3.2. *Structured work.*

(1) Write a program to test wave functions for an electron trapped in a one dimensional infinitely deep potential well of length, L, given by

$$L = 0.5 + \frac{5678}{10000} \text{ Å} \tag{70}$$

where 5678 are the last four digits of your student number.

(2) Use your program to check to see if it correctly determines the wavefunctions for the first three levels when their energies, E_1, E_2 and E_3, are entered as guesses into your program. Normalize the wavefunctions determined by the shooting method and compare them with the analytical wavefunctions for these levels.

(3) Modify your program so that it will automatically search for the first n energies and wavefunctions.

Remember, as noted above, to use a bisection method to find energy levels.

16.3.3. *Unstructured work.*

(1) Use the new program to find the wavefunctions for a particle trapped in a one dimensional harmonic well where the potential, $V(x)$, is given by

$$V(x) = \frac{1}{2} k x^2 \tag{71}$$

where k is analogous to the spring constant in simple harmonic motion.

(2) Explore different potential functions $V(x)$ and also the effect the mass of the particle has on the solutions. It would be interesting to explore potentials with a barrier, for example.

(3) Another interesting feature would be to explore unsymmetrical potentials, such as a Morse potential. This would require a different computational approach to the initial conditions and would be more challenging.

(4) Your report should include plots to show examples of different types of behaviour.

16.4. **Electric potential calculations including determination of the potential around a dipole.** *(return to contents)*

16.4.1. *Physics.* Potentials in space, V, must obey Laplace's equation where no charges are present;

$$\nabla^2 V = 0 \tag{72}$$

$$\frac{\partial^2 V}{\partial x^2} + \frac{\partial^2 V}{\partial y^2} + \frac{\partial^2 V}{\partial z^2} = 0 \tag{73}$$

In two dimensions this reduces to

$$\frac{\partial^2 V}{\partial x^2} + \frac{\partial^2 V}{\partial y^2} = 0 \tag{74}$$

If three dimensional space is divided up into a lattice of points with coordinates (i, j, k) then Laplace's equation can be satisfied by 'relaxing' the potential at each point repeatedly with the formula

$$\begin{aligned} V(i,j,k) = \tfrac{1}{6}(&V(i-1,j,k) + V(i+1,j,k) \\ &+ V(i,j-1,k) + V(i,j+1,k) \\ &+ V(i,j,k-1) + V(i,j,k+1) \end{aligned} \tag{75}$$

or in two dimensions

$$\begin{aligned} V(i,j,k) = \tfrac{1}{4}(&V(i-1,j,k) + V(i+1,j,k) \\ &+ V(i,j-1,k) + V(i,j+1,k) \end{aligned} \tag{76}$$

After all points have been relaxed enough times the potential will have reached a steady state that satisfies Laplace's equation and does not change when the relaxation formula is applied to all of its points. Note that the spacing between individual lattice points in x, y and z directions must be the same.

16.4.2. *Structured work.*

(1) Write a program to relax a two dimensional potential with an arbitrary lattice size.
(2) Use your programm to determine the potential for a two dimensional square space enclosed by four walls where three of the walls are held at ground ($V = 0$) and the forth wall is held at $V = 1$ V, where the unit V is Volts.
(3) Determine the potential for a two dimensional square space enclosed by four walls. The top and bottom wall should be set to ground potential ($V = 0$). The potentials on the left and right walls are determined from your student number. Te left wall should be set to a potential of $+56$ V and the right hand wall should be set to a potentials of -78 V, where 5678 are the last four digits of your student number. When you have calculated the potential you should plot and present a graph to show how the potential varies along a

straight line from the centre of the left hand wall to the centre of the right hand wall.

16.4.3. *Unstructured work.*

(1) Extend your program to work in three dimensions
(2) Consider a point charge at the centre of a cube and determine the potential between along a line from the centre of the cube to the centre of one of the faces of the cumbe. Compare this potential with the analytical formula for the potential surrounding a point charge. Note for the potential of the cell in the centre of the cube with the point charge should be set to the value expected at a distance, $dx/2$, from a charge, q, where dx is the spacing between cells and q is the charge of the point charge.
(3) Determine the potential surrounding a dipole in three dimensional space and compare the potential along a line of points equidistant from both of the charges of the dipole with the analytical expression for this potential.
(4) Determine different potentials for different charge distributions in two and three dimensional spaces.
(5) Another interesting feature would be to determine the motion of a charged particle through the potential, but this will be more challenging.
(6) Your report should include plots to show examples of different types of behaviour.

16.5. **Waves on a string including simulation of the propagation of a wave along a string and standing waves. Could include the use of an FFT - Fast Fourier Transform - to analyse the frequency components of the wave on the string.** *(return to contents)*

16.5.1. *Physics.* The physics of waves on a string depend on the following wave equation for motion;

$$\frac{\partial^2 y}{\partial t^2} = c^2 \frac{\partial^2 y}{\partial x^2} \tag{77}$$

where c is the speed of the wave that travels in the x direction causing displacement in the y direction. The speed, c, depends on the tension in the string, T, and the mass per unit length, μ;

$$c^2 = \frac{T}{\mu} \tag{78}$$

The partial differential wave equation 77 is difficult to handle as the displacement y is a function of both position x and time t. For numerical simulation it is necessary to split the string into segments and also to split time up into discrete steps.

First the string with displacement $y(x,t)$ is split into a number of segments with fixed positions, represented by $y_i(t)$, each separated by Δx. The right hand side of equation 77 can be approximated by

$$c^2\frac{\partial^2 y}{\partial x^2} \approx \frac{T}{\mu\Delta x^2}(y_{i-1} - 2y_i + y_{i+1}) \tag{79}$$

The $y_i(t)$ have a fixed position and so the partial differential on the left hand side of equation 77 can be approximated with

$$\frac{\partial^2 y}{\partial t^2} \approx \frac{d^2 y_i}{dt^2} \tag{80}$$

Now, Each segment also has a velocity, $v_i(t)$. and so the second order differential equation can be split into two first order differential equations and we obtain overall from equation 77 the following equations.

$$\frac{dy_i}{dt} = v_i \tag{81}$$

$$\frac{dv_i}{dt} \approx \frac{T}{\mu\Delta x^2}(y_{i-1} - 2y_i + y_{i+1}) \tag{82}$$

This is a computationally challenging project because it is easy for numerical issues to make the calculation unstable. The level of challenge here and in other projects will, of course, be taken into account when the projects are marked.

16.5.2. *Structured work.*

(1) Write a program to model the propogation of a wave on a string with an arbitrary length of string and arbitrary number of elements in the string.
(2) Use your model to determine the motion of a wave on a string with fixed ends. The wave should be started by making a 'Gaussian' shaped displacement in several of the elements of the string. Try using a maximum displacement from equilibrium of 5 mm. Use a tension of 100 N and mass per unit length of 0.01 kgm^{-1}. Use a string length, l, given by

$$l = 0.5 + \frac{5678}{10000} \text{ metres} \tag{83}$$

where 5678 are the last four digits of your student number. You should assume that all y_i and v_i values are zero except for the elements pulled out of place into the Gaussian shape, and these elements should start at rest with v_i values of zero. Plot graphs to show the progress of the wave at 0 ms, 2 ms, 4 ms, 8 ms and 12 ms after the Gaussian shape is pulled and released.
(3) Check that the speed of the wave along the string in your simulation is as expected from equation 78.

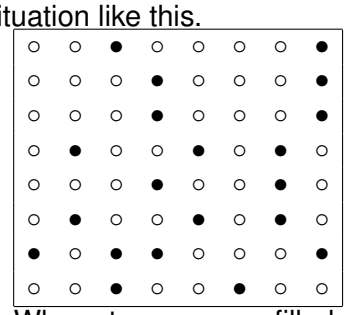
16.5.3. *Unstructured work.*

(1) In a more realistic pluck of a string one channel is pulled to 1 cm displacement and all the other elements of the string except for the fixed ends are also displaced so that the other elements lie on a straight line in between the ends and hte plucked element. Investigate the propogation of this wave where channel plucked is in the middle of the string and where the channel plucked is close to the end of the string.
(2) Find the effect of having string ends which are free to move up and down in the y direction.
(3) Modify your program so that T and μ can vary along the string and investigate the behaviour. It may be possible to simulate the behaviour of a whip.
(4) Investigate the frequencies of the waves excited on the string with Fast Fourier Transforms (FFTs).
(5) Your report should include plots to show examples of different types of behaviour.

16.6. **Percolation; investigation of open lattices where some sites are filled and some empty.** *(return to contents)*

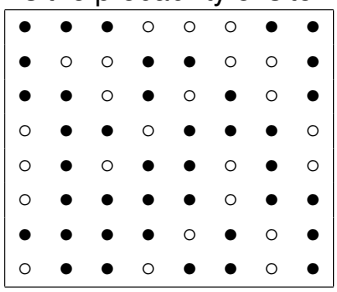
16.6.1. *Physics.* Percolation is concerned with lattice structures where some members of the lattice may be present and some members absent.

For example, if an 8 by 8 lattice in two dimensions is considered where the probability of a lattice site being filled are low then we may have a situation like this.



Where two or more filled sites touch each other they are said to form a cluster.

As the probability of site filling rises cluster size increases; for example



Now where a cluster extends from one side of the lattice to the other the cluster is said to percolate.

16.6.2. *Structured work.*

(1) Write a program to give random numbers between 0 and 1.
(2) Write a program to fill up a empty two dimensional 25 by 25 lattice at random locations. Note that you should use a single random number to determine both x and y values of the lattice position and not use onee random number to deteremine x and another random nunber to determine y.
(3) Determine the cluster size distribution as a function of site filling probability. Here several runs are required and the mean and standard deviation of cluster size should be reported
(4) Determine the fraction of filled sites at which percolation occurs. Do this for several runs. Determine whether this fraction changes with the size of the lattice.

16.6.3. *Unstructured work.*

(1) This project can be extended by changing the lattice size and by going into three dimensions. It would be interesting to repeat the points above for a three dimensional lattice.
(2) Your report should include plots to show examples of different types of behaviour.

16.7. **Percolation - immunization and disease spread.** *(return to contents)*

16.7.1. *Physics.* This project topic is closely related to the percolation described above. Here the lattice represents a population of people and the filled or empty lattice points represent people who are immunized or vulnerable to disease.

16.7.2. *Structured work.*

(1) Write a program to give random numbers between 0 and 1.
(2) Write a program to fill up a empty two dimensional 25 by 25 lattice at random locations. Note that you should fill the lattice points up one at a time with immunized indivuals. Note, be careful you always place an immunized individual on top of an unimmunnized individual as you add each immunized lattice point. Note that you should use a single random number to determine both x and y values of the lattice position and not use onee random number to deteremine x and another random nunber to determine y.
(3) Write a program to model the spread of a disease from a random starting point in the two dimensional lattice to any vulnerable lattice point that touches a diseased lattice point.

(4) Investigate the situation where the fraction of people immunized, p, is given by

$$p = 0.4 + \frac{5678}{50000} \tag{84}$$

where 5678 are the last four digits of your student number. You should perform at least 50 simulations and determine in each simulation the fraction of the unimmunized population who catch the disease if it is spread from one random unimmunized individual. Determine the mean fraction and standard deviation of the fraction who catch the disease. It would help to plot a histogram or other graph to show the probability distribution of different fractions of the population catching the disease with the immunization probability, p.

16.7.3. *Unstructured work.*

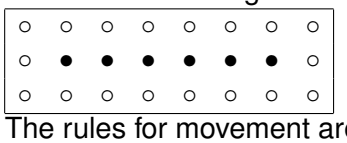
(1) When percolation occurs only part of the vulnerable population will catch the disease. As the fraction of filled (immunized) lattice spots increases determine the both the fraction of the total population that will catch the disease and also the fraction of the vulnerable population that will catch the disease.
(2) This project can be extended by changing the lattice size and by going into three dimensions. It would be interesting to repeat the points above for a three dimensional lattice.
(3) Your report should include plots to show examples of different types of behaviour.

16.8. **Protein folding including simple two dimensional simulations; could include three dimensional folding.** *(return to contents)*

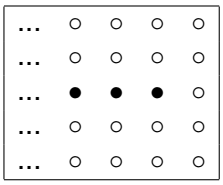
16.8.1. *Physics.* Protein folding is a fundamental complicated problem and here simple models of proteins will be considered.

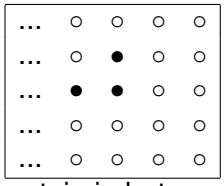
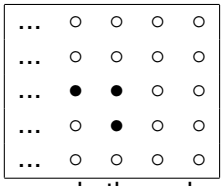
There are two main parts to this simulation of protein folding. The first part is the movement of the protein and the second part is the change in the energy of the protein as it folds and the selection of the lowest energy state.

Proteins are large complicated molecules made up of amino acids. For this project the structure of proteins is heavily simplified. The movement of the protein is going to be considered by starting with a linear chain of units in a two dimensional grid

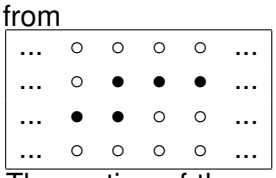
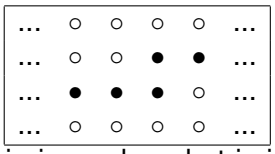
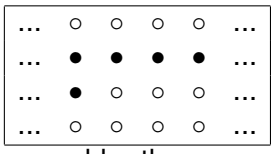


The rules for movement are realatively simple.

The ends are free to rotate into empty squares; thus, the end 

can move to either  or 

For parts of the protein in between the ends the only movement allowed is from

 to  or 

The motion of the protein is random, but is influenced by the energy of the molecule; in proteins weak bonds, such as 'hydrogen bonds' between proteins that are touching, but not adjacent to each other in the chain, hold the protein in its shape.

There are a number of ways of modelling this, but a key point is the temperature of the molecule. As parts of the protein touch each other the energy is reduced, but there has to be a chance of proteins moving away from each other. The probability of moving, p_{move} in this simplified model is given by

$$p_{move} = e^{\frac{-\Delta E}{kT}} \tag{85}$$

where ΔE is the energy lost or gained in making the move. It is negative if the molecule goes to a lower energy state and the probability is 1.

16.8.2. *Structured work.*

(1) Write a program to give random numbers between 0 and 1
(2) Write a program to model the movement of proteins. Before each move the program should determine all the different moves which are possible. One of the possible moves should be selected randomly. Demonstrate this for a protein 10 units long.
(3) When you have the protein moving randomly then make many moves of the protein and calculate the distance between the two ends of the protein after each step. Calculate the average distance between the two ends.

16.8.3. *Unstructured work.*

(1) Extend your program so that whenever two parts of the protein that are not linked touch the energy of the molecule drops by a quatity ΔE. Now if a move involves two parts of the protein moving apart the energy of the molecule will rise and there will be a probability of

moving that depends on the temperature and the probability p_{move}. Examine the long term behaviour of 10 unit proteins at different temperatures.

(2) There are other possible avenues to explore; protein length is one. Another avenue is the interaction between different units of the chain. In reality different amino acids will bind preferentially to particular other amino acids.

(3) Your report should include plots to show examples of different types of behaviour.

16.9. **Your own choice! This must be discussed prior to approval.** *(return to contents)*

Possible topics include

- Random Walks including self avoiding walks in two and three dimensions and diffusion.
- Cluser growth including Eden growth model, diffusion-limited aggregation and the determination of fractal dimension. Could include work in two and three dimensions
- Simulation of traffic on a motorway.

[This page is intentionally Blank]

17. REFERENCE TABLES

(return to contents)

in the table below, the intial values are `a` = 5, `b` = 3, `c` = 13

Operation	Example	Description	final value of c
+	c=a+b	c takes value of a+b	8
+=	c+=b	c takes value of c+b	16
-	c=a-b	c takes value of a-b	2
-=	c-=b	c takes value of c-b	10
*	c=a*b	c takes value of a $\times$ b	15
*=	c*=b	c takes value of c $\times$ b	39
/	c=b/a	c takes value of b/a	0.6
/=	c/=a	c takes value of c/a	2.6
**	c=a**b	c takes value of a^b	125
**=	b**=a	b takes value of b^a	243
//	c=a//b	c takes value of *floor division* of a by b	1
//=	c//=b	c takes value of *floor division* of c by b	4
%	c=a%b	c takes value of a *modulo* b	2
%=	c%=b	c takes value of c *modulo* b	1

Function	Description	Function	Description
math.sqrt(x)	$\sqrt{x}$	math.pow(a,b)	a^b
math.sin(x)	$\sin(x)$	math.asin(x)	$\sin^{-1}(x)$
math.cos(x)	$\cos(x)$	math.acos(x)	$\cos^{-1}(x)$
math.tan(x)	$\tan(x)$	math.atan(x)	$\tan^{-1}(x)$
math.degrees(x)	$180x/\pi$	math.radians (x)	$\pi x/180$
math.pi	π (=3.141...)	math.e	e (=2.718...)
math.exp(x)	e^x	math.expm1(x)	$\mathrm{e}^x - 1$
math.log(x)	$\ln x = \log_{\mathrm{e}}$ x	math.log10(x)	$\log_{10} x$
math.log(a,b)	$\log_{\mathrm{b}} a$	math.log2(x)	$\log_2 x$

The following operations can be put in logic statements;

Operation	Description	Operation	Description
$<$	less than	$<=$	less than or equal to
$>$	greater than	$>=$	greater than or equal to
==	equal to	!=	not equal to
and	logical and	or	logical or
not	logical not	(...)	establish priority

function	number range	output with a=7, b=3, c=2
range(a)	zero to a-1	0,1,2,3,4,5,6
range(b,a)	b to a-1	3,4,5,6
range(b,a,c)	b to a-1 in steps of c	3,5

CENTRE FOR PLASMA PHYSICS, SCHOOL OF MATHEMATICS AND PHYSICS, QUEEN'S UNIVERSITY, BELFAST, BT7 1NN, U.K.

Email address: t.field @ qub.ac.uk --v5.0